# A CONTINUOUS-TIME ACTOR-CRITIC FRAMEWORK FOR JUMP-DIFFUSION CONTROLS WITH NORMALIZING FLOWS*

LIYA GUO†, RUIMENG HU‡, XU YANG§, AND YI ZHU¶

**Abstract.** Continuous-time stochastic control with time-inhomogeneous jump–diffusion dynamics is central in finance and economics, but computing optimal policies is difficult under explicit time dependence, discontinuous shocks, and high dimensionality. We propose an actor–critic framework that serves as a mesh-free solver for entropy-regularized control problems and stochastic games with jumps. The approach is built on a time-inhomogeneous "little" $q$-function and an appropriate occupation measure, yielding a policy-gradient representation that accommodates time-dependent drift, volatility, and jump terms. To represent expressive stochastic policies in continuous-action spaces, we parameterize the actor using conditional normalizing flows, enabling flexible non-Gaussian policies while retaining exact likelihood evaluation for entropy regularization and policy optimization. We validate the method on time-inhomogeneous linear–quadratic control, Merton portfolio optimization, and a multi-agent portfolio game, using explicit solutions or high-accuracy benchmarks. Numerical results demonstrate stable learning under jump discontinuities, accurate approximation of optimal stochastic policies, and favorable scaling with respect to dimension and number of agents.

**Key words.** Jump diffusion, Stochastic Control, Reinforcement Learning, Hamilton–Jacobi–Bellman Equation, Actor-Critic Methods, Normalizing Flow

**MSC codes.** 93E20, 60H10, 65C30, 91G10

## 1. Introduction.

Continuous-time stochastic control provides a mathematical framework for dynamical decision making in finance and economics [33]. Many problems such as portfolio selection [13, 29] can be formulated as controlling stochastic differential equations to maximize (or minimizing) an expected discounted objective. From a computational standpoint, however, classical approaches based on dynamic programming or stochastic maximum principles become difficult to implement when the state dimension is large [11], and when the underlying dynamics are unknown or only partially specified. These challenges have motivated the development of continuous-time reinforcement learning (RL) methods [24, 25, 39, 40] that combined with neural networks, aiming to learn near-optimal control directly from interaction with the environment without requiring explicit model structure and with improved scalability to higher dimensions.

A growing literature has developed continuous-time analogs of policy evaluation and policy improvement. For policy evaluation, temporal-difference type schemes are derived in [15, 24], providing practical methods for approximating value function directly in continuous time. For policy improvement, [25] exploits martingale structure to rewrite policy-gradient objectives as policy-evaluation problems, yielding imple-

*Submitted to the editors DATE.

**Funding:** Y.Z. and L.G. were partially supported by the National Key R&D Program of China (grant 2021YFA0719200). R.H. was partially supported by the ONR grant N00014-24-1-2432, the Simons Foundation (MP-TSM-00002783), and the NSF grant DMS-2420988.

†Yau Mathematical Sciences Center, Tsinghua University, Beijing 100084, China; and Department of Mathematics, Tsinghua University, Beijing 100084, China (gly22@mails.tsinghua.edu.cn).

‡Department of Mathematics, and Department of Statistics and Applied Probability, University of California, Santa Barbara, CA 93106-3080, USA (rhu@ucsb.edu).

§Department of Mathematics, University of California, Santa Barbara, CA 93106-3080, USA (xuyang@math.ucsb.edu).

¶Yau Mathematical Sciences Center, Tsinghua University, Beijing 100084, China; and Yanqi Lake Beijing Institute of Mathematical Sciences and Applications, Beijing 101408, China (yizhu@tsinghua.edu.cn).

mentable update rules. From an action-value viewpoint, [26] studies continuous-time $q$ learning and introduces a first-order surrogate, the "little" $q$-function, to avoid the degeneracy of the conventional "big" $Q$-function in the continuous-time limit [38]. Most of these developments are focused on finite-horizon criteria, with related extensions to mean-field control in [41]. Infinite-horizon continuous-time policy-gradient formulas have appeared more recently, for example in [42].

In many financial settings, pure diffusion models are inadequate because asset prices and economic factors may exhibit abrupt movements driven by liquidity shocks or macroeconomic events [3, 31]. Incorporating jumps is therefore essential for capturing heavy tails, discontinuities, and jump risk premia for markets [4, 6]. This has motivated a growing body of work on deep learning and RL for jump-diffusion dynamics [9, 12, 14, 18, 19, 27, 28]. Motivated by the little-$q$ methodology, [9, 18] extend continuous-time q-learning ideas to stochastic policies and entropy regularization in jump-diffusion settings, while [28] develops an actor-critic method for deterministic controls in finite-horizon jump-diffusion games, and [14] considers optimal switching problems under jump dynamics.

Most existing formulations, however, remain tied to finite-horizon objectives and often adopt Gaussian policy parameterizations in practice. In many situations, the optimal stochastic policy is non-Gaussian; see, for example, [9]. This paper addresses these gaps by developing a learning framework for discounted infinite-horizon control of time-inhomogeneous jump-diffusions under general stochastic (possibly non-Gaussian) policies. Our first contribution is the introduction of a continuous-time little $q$-function and a time-dependent discounted occupation measure, and the establishment of structural properties that connect these objects to policy improvement. These results lead to a policy-gradient representation valid for general time-inhomogeneous jump-diffusions on an infinite horizon. For convenience, we provide a comparative summary of continuous-time $q$-function formulations in Table 1.

Table 1: Comparison continuous-time $Q/q$-function formulations.

| Work | Time setting | State dynamics | Learning objects | Role of $q$ function | Policy class |
|---|---|---|---|---|---|
| [26] | Continuous, finite horizon | Diffusion | $V_\psi(t,\boldsymbol{x})$, $q_\theta(t,\boldsymbol{x},\boldsymbol{u})$ | $\pi^*(\boldsymbol{u}\mid t,\boldsymbol{x}) \propto \exp\left\{\frac{1}{\gamma}q^*(t,\boldsymbol{x},\boldsymbol{u})\right\}$ | Gaussian |
| [9, 18] | | Jump-diffusion | $V_\psi(t,\boldsymbol{x})$, $q_\theta(t,\boldsymbol{x},\boldsymbol{u})$ | $q$ Learned via martingale orthogonality | |
| [42] | Continuous, time-homog., infinite horizon | Diffusion | $V_\psi(\boldsymbol{x})$, $\pi_\theta(\boldsymbol{x})$ | Policy gradient thm for $\pi^*$ $q$ approximated by GAE | Gaussian |
| This work | Continuous, time-inhomog., infinite horizon | Jump-diffusion | $V_\psi(t,\boldsymbol{x})$, $\pi_\theta(t,\boldsymbol{x})$ | Policy gradient thm for $\pi^*$ $q$ approximated by GAE | General (normalizing flow) |

Our second contribution is to provide tractable benchmarks by deriving explicit solutions in several canonical specifications, including linear-quadratic control, the Merton problem with jumps, and multi-agent games with jump-driven CARA utilities, together with representative time-inhomogeneous variants. These closed-form policies serve as ground truth for assessing numerical accuracy. Finally, we propose an implementable actor–critic algorithm that combines the derived policy-gradient representation with a conditional normalizing-flow parameterization of stochastic policies. The flow-based construction enables expressive, non-Gaussian distributions for controls while preserving tractable likelihoods and gradients, which is essential in the

presence of entropy regularization and policy-gradient-type theorems. Numerical experiments in both low- and high-dimensional regimes demonstrate stable learning behavior across a range of time-dependent jump-diffusion models.

The rest of the paper is organized as follows. Section 2 introduces the problem setting, including the classical jump-diffusion stochastic control problem and its entropy-regularized formulation. Section 3 presents the proposed actor-critic framework: Section 3.1 develops policy evaluation for the critic, Section 3.2 introduces the "little" $q$-function and the occupation measure, and develops policy improvement for the actor with its theoretical justification, and Section 3.3 details the conditional normalizing flow parameterization for the actor. Section 4 states explicit solutions for several canonical problems and reports numerical experiments. We conclude in Section 5.

**2. Problem Setup.** Let $(\Omega, \mathcal{F}, \mathbb{F} := (\mathcal{F}_t)_{t\geq 0}, \mathbb{P})$ be a filtered probability space satisfying the usual conditions. Let $\boldsymbol{W} = (\boldsymbol{W}_t)_{t\geq 0}$ be a $d$-dimensional Brownian motion, $N(\mathrm{d}t, \mathrm{d}\boldsymbol{z})$ be a Poisson random measure corresponding to a Lévy process $(\boldsymbol{L}_t)_{t\geq 0}$, and $\nu$ be the Lévy measure on $\mathbb{R}^d$ satisfying the integrability condition $\int_{\mathbb{R}^d} \min\{|\boldsymbol{z}|^2, 1\}\nu(\mathrm{d}\boldsymbol{z}) < \infty$ [16]. The associated compensated Poisson random measure is defined as $\tilde{N}(\mathrm{d}t, \mathrm{d}\boldsymbol{z}) := N(\mathrm{d}t, \mathrm{d}\boldsymbol{z}) - \nu(\mathrm{d}\boldsymbol{z})\,\mathrm{d}t$ and we assume that $\boldsymbol{W}$ and $N$ are independent.

We first recall the classical strict-control formulation. Let $\boldsymbol{u} = (\boldsymbol{u}_t)_{t\geq 0}$ be an admissible progressively measurable control process taking values in $\mathcal{A} \subset \mathbb{R}^m$. The controlled state process $\boldsymbol{X} = (\boldsymbol{X}_t)_{t\geq 0} \in \mathbb{R}^d$ follows the Itô–Lévy SDE

$$\mathrm{d}\boldsymbol{X}_t = \boldsymbol{b}(t, \boldsymbol{X}_{t-}, \boldsymbol{u}_t)\,\mathrm{d}t + \boldsymbol{\sigma}(t, \boldsymbol{X}_{t-}, \boldsymbol{u}_t)\,\mathrm{d}\boldsymbol{W}_t + \int_{\mathbb{R}^d} \boldsymbol{\alpha}(t, \boldsymbol{X}_{t-}, \boldsymbol{u}_t, \boldsymbol{z})\,\tilde{N}(\mathrm{d}t, \mathrm{d}\boldsymbol{z})\,. \tag{2.1}$$

For a given admissible control process, this SDE is well posed under the usual Lipschitz, linear-growth, and jump-integrability conditions [31].

In reinforcement learning, it is natural to consider randomized (stochastic) policies, under which the action at $(t, \boldsymbol{x})$ is drawn from a state-dependent distribution $\pi(\cdot \mid t, \boldsymbol{x}) \in \mathcal{P}(\mathcal{A})$. However, as discussed in [23, 37], a measurability issue arises: to make the above SDE well posed under randomized policy, one needs a process $\boldsymbol{u}$ that is $\mathcal{F}$-progressively measurable and satisfies $\boldsymbol{u}_t \mid (\boldsymbol{X}_t = \boldsymbol{x}) \sim \pi(\cdot \mid t, \boldsymbol{x})$ for each $t$. Such a construction is not immediate on a fixed stochastic basis, since time is uncountable and one cannot literally "sample independently at every instant". To avoid this issue, following [23], we work with the exploratory state process

$$\mathrm{d}\boldsymbol{X}^\pi_t = \tilde{\boldsymbol{b}}(t, \boldsymbol{X}^\pi_{t-}; \pi)\,\mathrm{d}t + \tilde{\boldsymbol{\sigma}}(t, \boldsymbol{X}^\pi_{t-}; \pi)\,\mathrm{d}\boldsymbol{W}_t + \int_{\mathbb{R}^d\times[0,1]^m} \boldsymbol{\alpha}\big(t, \boldsymbol{X}^\pi_{t-},\ G^\pi(t, \boldsymbol{X}^\pi_{t-}, \boldsymbol{r}),\ \boldsymbol{z}\big)\,\tilde{N}(\mathrm{d}t, \mathrm{d}\boldsymbol{z}, \mathrm{d}\boldsymbol{r}), \tag{2.2}$$

where $G^\pi : [0,\infty)\times\mathbb{R}^d\times[0,1]^m \to \mathcal{A}$ is a measurable function such that $(G^\pi(t, \boldsymbol{x}, \cdot))_{\#}\mathcal{U} = \pi(\cdot \mid t, \boldsymbol{x})$, when $\mathcal{U}$ is the Lebesgue probability measure on $[0,1]^m$, $N(\mathrm{d}t, \mathrm{d}\boldsymbol{z}, \mathrm{d}\boldsymbol{r})$ is a Poisson random measure on $(0,\infty)\times\mathbb{R}^d\times[0,1]^m$ with compensator $\nu(\mathrm{d}\boldsymbol{z})\,\mathcal{U}(\mathrm{d}\boldsymbol{r})\,\mathrm{d}t$, independent of the Brownian motion $\boldsymbol{W}$, $\tilde{N}(\mathrm{d}t, \mathrm{d}\boldsymbol{z}, \mathrm{d}\boldsymbol{r}) := N(\mathrm{d}t, \mathrm{d}\boldsymbol{z}, \mathrm{d}\boldsymbol{r}) - \nu(\mathrm{d}\boldsymbol{z})\,\mathcal{U}(\mathrm{d}\boldsymbol{r})\,\mathrm{d}t$ is the compensated measure, and $\tilde{\boldsymbol{b}}, \tilde{\boldsymbol{\Sigma}}$ are defined as

$$\tilde{\boldsymbol{b}}(t, \boldsymbol{x}; \pi) := \int_{\mathcal{A}} \boldsymbol{b}(t, \boldsymbol{x}, \boldsymbol{u})\,\pi(\boldsymbol{u} \mid t, \boldsymbol{x})\mathrm{d}\boldsymbol{u},\ \ \tilde{\boldsymbol{\Sigma}} = \tilde{\boldsymbol{\sigma}}^2(t, \boldsymbol{x}; \pi) := \int_{\mathcal{A}} \boldsymbol{\sigma}(t, \boldsymbol{x}, \boldsymbol{u})\boldsymbol{\sigma}(t, \boldsymbol{x}, \boldsymbol{u})^\top\,\pi(\boldsymbol{u} \mid t, \boldsymbol{x})\mathrm{d}\boldsymbol{u}\,, \tag{2.3}$$

For each admissible policy $\pi$, we assume that $\tilde{\boldsymbol{b}}(\cdot, \cdot; \pi)$, $\tilde{\boldsymbol{\sigma}}(\cdot, \cdot; \pi)$, and $(t, \boldsymbol{x}, \boldsymbol{z}, \boldsymbol{r}) \mapsto \boldsymbol{\alpha}(t, \boldsymbol{x}, G^\pi(t, \boldsymbol{x}, \boldsymbol{r}), \boldsymbol{z})$ satisfy the standard Lipschitz and linear-growth conditions, with

those for the jump coefficient understood in $L^2(\nu \otimes \mathcal{U})$. Then (2.2) admits a unique strong solution; see [18, Proposition 1]. Note that the exploratory SDE is an averaged continuous-time representation of the randomized policy and is not, in general, the directly observable state trajectory generated by a single realization of sampled actions. In the numerical implementation, actions are instead sampled on a discrete-time grid and held fixed between consecutive grid points. The resulting grid-sampled dynamics is described in Section S.3.1 (supplementary materials). Under the assumptions of [7], it converges weakly, as the grid size tends to zero, to a limiting process corresponding in law to the exploratory SDE (2.2).

With the exploratory state dynamics (2.2), we consider the following entropy-regularized reward:

$$\tilde{f}(s, \boldsymbol{y}; \pi) := \int_{\mathcal{A}} \big(f(s, \boldsymbol{y}, \boldsymbol{u}) - \gamma \log \pi(\boldsymbol{u} \mid s, \boldsymbol{y})\big)\, \pi(\boldsymbol{u} \mid s, \boldsymbol{y}) \mathrm{d}\boldsymbol{u}, \tag{2.4}$$

where $f$, continuous at $(s, \boldsymbol{y})$ and locally uniformly integrable along $\boldsymbol{y}$, is the standard running reward. We consider $\mathcal{S}\big(\pi(\cdot \mid s, \boldsymbol{y})\big) := -\int_{\mathcal{A}} \pi(\boldsymbol{u} \mid s, \boldsymbol{y}) \log \pi(\boldsymbol{u} \mid s, \boldsymbol{y})\, \mathrm{d}\boldsymbol{u}$, the Shannon entropy, which encourages randomized policies and may improve numerical stability. For long-term control, let $\beta > 0$ be a discount factor. We then define the entropy-regularized expected discounted reward by

$$J(t, \boldsymbol{x}; \pi) := \mathbb{E}\Big[\int_t^\infty e^{-\beta(s-t)} \tilde{f}\big(s, \boldsymbol{X}_s^\pi; \pi\big)\, \mathrm{d}s \mid \boldsymbol{X}_t^\pi = \boldsymbol{x}\Big], \tag{2.5}$$

where $\boldsymbol{X}_s^\pi$ solves the exploratory dynamics (2.2), and $\gamma \geq 0$ characterizes the intensity of regularization.

For a fixed policy $\pi$, the function $J(t, \boldsymbol{x}; \pi)$ satisfies (cf. [25, Lemma 3])

$$0 = \partial_t J(t, \boldsymbol{x}; \pi) + \mathcal{L}^\pi J(t, \boldsymbol{x}; \pi) + \tilde{f}(t, \boldsymbol{x}; \pi) - \beta J(t, \boldsymbol{x}; \pi), \tag{2.6}$$

where $\mathcal{L}^\pi$ is the $\pi$-averaged infinitesimal generator

$$(\mathcal{L}^\pi \varphi)(t, \boldsymbol{x}) := \int_{\mathcal{A}} (\mathcal{L}^{\boldsymbol{u}} \varphi)(t, \boldsymbol{x})\, \pi(\boldsymbol{u} \mid t, \boldsymbol{x}) \mathrm{d}\boldsymbol{u}, \quad \varphi \in C_c^{1,2}([0, T] \times \mathbb{R}^d),\ \forall\, T > 0\,, \tag{2.7}$$

and $\mathcal{L}^{\boldsymbol{u}}$ is the generator for fixed control sampling

$$\begin{aligned}(\mathcal{L}^{\boldsymbol{u}} \varphi)(t, \boldsymbol{x}) &= \boldsymbol{b}(t, \boldsymbol{x}, \boldsymbol{u}) \cdot \nabla_{\boldsymbol{x}} \varphi(t, \boldsymbol{x}) + \tfrac{1}{2} \operatorname{Tr}\big(\boldsymbol{\sigma}(t, \boldsymbol{x}, \boldsymbol{u}) \boldsymbol{\sigma}(t, \boldsymbol{x}, \boldsymbol{u})^\top \nabla_{\boldsymbol{x}}^2 \varphi(t, \boldsymbol{x})\big) \\ &\quad + \int_{\mathbb{R}^d} \big(\varphi\big(t, \boldsymbol{x} + \boldsymbol{\alpha}(t, \boldsymbol{x}, \boldsymbol{u}, \boldsymbol{z})\big) - \varphi(t, \boldsymbol{x}) - \boldsymbol{\alpha}(t, \boldsymbol{x}, \boldsymbol{u}, \boldsymbol{z}) \cdot \nabla_{\boldsymbol{x}} \varphi(t, \boldsymbol{x})\big)\, \nu(\mathrm{d}\boldsymbol{z}).\end{aligned} \tag{2.8}$$

The optimal value function is

$$V(t, \boldsymbol{x}) := \sup_\pi J(t, \boldsymbol{x}; \pi) = \sup_\pi \mathbb{E}\Big[\int_t^\infty e^{-\beta(s-t)}\, \tilde{f}\big(s, \boldsymbol{X}_s^\pi; \pi\big)\, \mathrm{d}s \mid \boldsymbol{X}_t^\pi = \boldsymbol{x}\Big], \tag{2.9}$$

which satisfies the entropy-regularized Hamilton-Jacobi-Bellman (HJB) equation by dynamic programming (cf. [31])

$$\begin{aligned}0 = \partial_t V(t, \boldsymbol{x}) + \sup_{\pi(\cdot \mid t, \boldsymbol{x})} \int_{\mathcal{A}} \Big[&\mathscr{H}\big(t, \boldsymbol{x}, \boldsymbol{u}, V(t, \boldsymbol{x}), \nabla_{\boldsymbol{x}} V(t, \boldsymbol{x}), \nabla_{\boldsymbol{x}}^2 V(t, \boldsymbol{x})\big) \\ &- \gamma \log \pi(\boldsymbol{u} \mid t, \boldsymbol{x})\Big]\, \pi(\boldsymbol{u} \mid t, \boldsymbol{x}) \mathrm{d}\boldsymbol{u} - \beta V(t, \boldsymbol{x}).\end{aligned} \tag{2.10}$$

Here $\mathscr{H} : [0,\infty) \times \mathbb{R}^d \times \mathcal{A} \times \mathbb{R} \times \mathbb{R}^d \times \mathbb{S}^d \to \mathbb{R}$ is the Hamiltonian defined by

$$\begin{aligned}\mathscr{H}(t,\boldsymbol{x},\boldsymbol{u},&V(t,\boldsymbol{x}),\nabla_{\boldsymbol{x}}V(t,\boldsymbol{x}),\nabla^2_{\boldsymbol{x}}V(t,\boldsymbol{x})) := f(t,\boldsymbol{x},\boldsymbol{u}) + \boldsymbol{b}(t,\boldsymbol{x},\boldsymbol{u})\cdot\nabla_{\boldsymbol{x}}V(t,\boldsymbol{x})\\ &+\tfrac{1}{2}\operatorname{Tr}\big(\boldsymbol{\sigma}(t,\boldsymbol{x},\boldsymbol{u})\boldsymbol{\sigma}(t,\boldsymbol{x},\boldsymbol{u})^\top\nabla^2_{\boldsymbol{x}}V(t,\boldsymbol{x})\big)\\ &+\int_{\mathbb{R}^d}\big(V\big(t,\boldsymbol{x}+\boldsymbol{\alpha}(t,\boldsymbol{x},\boldsymbol{u},\boldsymbol{z})\big)-V(t,\boldsymbol{x})-\boldsymbol{\alpha}(t,\boldsymbol{x},\boldsymbol{u},\boldsymbol{z})\cdot\nabla_{\boldsymbol{x}}V(t,\boldsymbol{x})\big)\,\nu(\mathrm{d}\boldsymbol{z})\,,\end{aligned} \tag{2.11}$$

and $\mathbb{S}^d$ denotes the space of real $d\times d$ symmetric matrices. Therefore, the optimal policy $\pi^*$ is solved as a maximizer of the sup part in (2.10).

Note that when $\gamma = 0$ in (2.4), the stochastic control problem reduces to the standard problem with deterministic control. In this case, the state process is governed by the classical controlled Itô–Lévy SDE with an admissible progressively measurable control process $\boldsymbol{u} = (\boldsymbol{u}_s)_{s\geq t}$, taking values in $\mathcal{A}\subset\mathbb{R}^m$, and the associated HJB equation becomes

$$0 = \partial_t V(t,\boldsymbol{x}) + \sup_{\boldsymbol{u}\in\mathcal{A}}\big\{(\mathcal{L}^{\boldsymbol{u}}V)(t,\boldsymbol{x}) + f(t,\boldsymbol{x},\boldsymbol{u})\big\} - \beta V(t,\boldsymbol{x})\,, \tag{2.12}$$

where the generator $\mathcal{L}^{\boldsymbol{u}}$ is defined as (2.8).

**3. Actor-Critic for Time Inhomogeneous Jump Diffusion Control.** We solve the infinite-horizon stochastic control problem via reinforcement learning (RL) using an actor-critic framework [5]. In RL, the *actor* usually refers to the randomized policy $\pi$ and the *critic* refers to the value $J(\cdot;\pi)$, used to evaluate the goodness of the current policy $\pi$. The actor-critic method consists of two steps: policy evaluations for the critic and policy improvement for the actor. By performing them interactively, one hopes to reach the optimal policy and value function $V$.

Most existing continuous-time RL work is developed for finite horizons and/or dynamics driven only by Brownian motions [24–26, 39]. In the infinite-horizon setting considered here, the policy update step cannot be reduced to maximizing a finite-interval objective in the same manner as in finite-horizon formulations [28]. This necessitates a policy improvement principle that is consistent with discounting and time inhomogeneity, and remains valid under jump-diffusion dynamics. The resulting actor update scheme is developed in Section 3.2, while the critic is introduced below.

**3.1. Critic: Policy Evaluation.** Consider the critic $V_\psi$, parameterized by $\psi$. Our goal is to learn an accurate value approximation from incremental samples, without explicitly solving the partial integro-differential equation (PIDE) (2.6). To this end, we use the continuous-time Bellman principle [24], which leads to temporal-difference learning: we update the critic by minimizing TD errors computed along sampled trajectories.

**Bellman equation and TD error.** Fix a stochastic feedback policy $\pi$. Recall the entropy-regularized performance functional $J$ in (2.5) and the entropy-regularized reward $\tilde{f}(\cdot;\pi)$ in (2.4). For any fixed deterministic $\delta_t > 0$, by the law of total expectation and the Markov property under $\pi$, the discounted performance $J$ satisfies the Bellman equation

$$\begin{aligned}J(t,\boldsymbol{X}^\pi_t;\pi) = \mathbb{E}\Big[\int_t^{t+\delta_t} e^{-\beta(s-t)}\int_{\mathcal{A}}\big(f(s,\boldsymbol{X}^\pi_{s-},\boldsymbol{u}) - \gamma\log\pi(\boldsymbol{u}\mid s,\boldsymbol{X}^\pi_{s-})\big)\\ \pi(\boldsymbol{u}\mid s,\boldsymbol{X}^\pi_{s-})\mathrm{d}\boldsymbol{u}\,\mathrm{d}s + e^{-\beta\delta_t}\,J(t+\delta_t,\boldsymbol{X}^\pi_{t+\delta_t};\pi)\;\Big|\;\mathcal{F}_t\Big].\end{aligned} \tag{3.1}$$

We then define the one-step TD error over $[t, t+\delta_t)$ by

$$\delta^t_{\mathrm{TD}} := \int_t^{t+\delta_t} e^{-\beta(s-t)} \int_{\mathcal{A}} \big(f(s, \boldsymbol{X}^\pi_{s-}, \boldsymbol{u}) - \gamma \log \pi(\boldsymbol{u} \mid s, \boldsymbol{X}^\pi_{s-})\big)\, \pi(\boldsymbol{u} \mid s, \boldsymbol{X}^\pi_{s-}) \mathrm{d}\boldsymbol{u}\, \mathrm{d}s$$
$$+ e^{-\beta\delta_t} V_\psi(t+\delta_t, \boldsymbol{X}^\pi_{t+\delta_t}; \pi) - V_\psi(t, \boldsymbol{X}^\pi_t; \pi), \tag{3.2}$$

and (3.1) implies that, for the critic $V_\psi$ that evaluates $J$ exactly, $\mathbb{E}\big[\delta^t_{\mathrm{TD}} \mid \mathcal{F}_t\big] = 0$.

**Martingale-corrected TD error.** By Itô's formula, the last two terms in the one-step TD error admit the decomposition

$$e^{-\beta\delta_t} V_\psi(t+\delta_t, \boldsymbol{X}^\pi_{t+\delta_t}; \pi) - V_\psi(t, \boldsymbol{X}^\pi_t; \pi)$$
$$= \int_t^{t+\delta_t} e^{-\beta(s-t)} \big(\partial_s V_\psi + \mathcal{L}^\pi V_\psi - \beta V_\psi\big)(s, \boldsymbol{X}^\pi_{s-}; \pi)\, \mathrm{d}s \;+\; \mathcal{I}^\pi_{t,t+\delta_t}.$$

where $\mathcal{I}^\pi_{t,t+\delta_t}$ defined as

$$\mathcal{I}^\pi_{t,t+\delta_t} := \int_t^{t+\delta_t} e^{-\beta(s-t)} \big(\boldsymbol{\sigma}(s, \boldsymbol{X}^\pi_{s-}, \boldsymbol{u}_s)^\top \nabla_x V_\psi(s, \boldsymbol{X}^\pi_{s-}; \pi)\big)^\top \mathrm{d}\boldsymbol{W}_s$$
$$+ \int_t^{t+\delta_t} \int_{\mathbb{R}^d} e^{-\beta(s-t)} \big[V_\psi\big(s, \boldsymbol{X}^\pi_{s-} + \boldsymbol{\alpha}(s, \boldsymbol{X}^\pi_{s-}, \boldsymbol{u}_s, \boldsymbol{z}); \pi\big) - V_\psi(s, \boldsymbol{X}^\pi_{s-}; \pi)\big] \tilde{N}(\mathrm{d}s, \mathrm{d}\boldsymbol{z}), \tag{3.3}$$

captures the instantaneous fluctuations induced by Brownian and jump noises. As discussed in [28, 43, 44], it has mean zero but adds extra variance to the learning signal. Therefore, subtracting $\mathcal{I}^\pi_{t,t+\delta_t}$ from the one-step TD error reduces variance while preserving unbiasedness.

Accordingly, we define the martingale-corrected TD error by:

$$\tilde{\delta}^t_{\mathrm{TD}} := \delta^t_{\mathrm{TD}} - \mathcal{I}^\pi_{t,t+\delta_t} = \textstyle\int_t^{t+\delta_t} e^{-\beta(s-t)} \big(\tilde{f}(s, \boldsymbol{X}^\pi_{s-}; \pi) + \partial_s V_\psi + \mathcal{L}^\pi V_\psi - \beta V_\psi\big)(s, \boldsymbol{X}^\pi_{s-}; \pi)\, \mathrm{d}s. \tag{3.4}$$

If the critic $V_\psi$ evaluates $J$ exactly, i.e., it solves the PIDE (2.6), then $\tilde{\delta}^t_{\mathrm{TD}} = 0$ a.s..

**3.2. Actor: Policy Improvement.** Policy improvement updates the actor using the critic's value information to increase the expected discounted reward. A popular approach is policy gradient: the actor is parameterized (for example by neural networks), and the critic is used to construct an estimator of the gradient of the objective with respect to the actor parameters, which then drives the actor update.

In discrete-time with state/action space, the action-value $Q$ function is a common choice for this purpose. In continuous time and state/action space, however, a direct analogue of discrete-time $Q$-learning is intrinsically delicate: the standard ("capital" $Q$) action-value function degenerates to the value function, and naïve discretization-based updates can be highly sensitive to the time step [26, 35]. These issues motivate the "little" $q$-formulation advocated in [9, 26, 42].

Following this line, we introduce a time-inhomogeneous "little" $q$-function for infinite-horizon jump-diffusion control (Section 3.2.1) and derive the corresponding policy gradient theorem (Theorem 3.4 in Section 3.2.2). Because the resulting gradient depends on $q(\cdot; \pi)$ and is not directly implementable from data, we adopt a generalized advantage estimator justified by Lemma 3.5. This extension for time-inhomogeneous is nontrivial: unlike the critic, which extends by standard time augmentation, the actor update is not a mere "add $t$" modification, since explicit time dependence interacts with discounting and reshapes the occupation measure on $[t, \infty) \times \mathbb{R}^d$; this is why the $\frac{\partial J}{\partial t}$ term must enter $q$.

**3.2.1. Occupation Measure and $q$-Function.** To accommodate possible time inhomogeneity in the infinite-horizon setting, we first define a discounted occupation measure on $[t,\infty)\times\mathbb{R}^d$, extending [42, Def. 2]. This definition is the $\beta$-potential of $\boldsymbol{X}^\pi$ and characterizes the discounted visitation frequencies of the time-state process starting from $(t,\boldsymbol{x})$.

DEFINITION 3.1. *Fix $\beta>0$. Let $\pi$ be an admissible stochastic policy such that, for every $(t,\boldsymbol{x})\in[0,\infty)\times\mathbb{R}^d$, the exploratory dynamics (2.2) starting from $\boldsymbol{X}_t^\pi=\boldsymbol{x}$ admits a unique strong solution $(\boldsymbol{X}_s^\pi)_{s\geq t}$. The $\beta$-discounted occupation measure of $\boldsymbol{X}^\pi$ is defined by*

$$\mu^{\pi,t,\boldsymbol{x}}(A):=\mathbb{E}\Big[\int_t^\infty e^{-\beta(s-t)}\,\mathbf{1}_{\{(s,\boldsymbol{X}_s^\pi)\in A\}}\,\mathrm{d}s\Big],\qquad A\in\mathcal{B}([t,\infty)\times\mathbb{R}^d). \tag{3.5}$$

*This is a finite measure on $[t,\infty)\times\mathbb{R}^d$ with total mass $\mu^{\pi,t,\boldsymbol{x}}\big([t,\infty)\times\mathbb{R}^d\big)=\mathbb{E}[\int_t^\infty e^{-\beta(s-t)}\,\mathrm{d}s]=\int_t^\infty e^{-\beta(s-t)}\,\mathrm{d}s=\beta^{-1}$. Unless otherwise stated, expectations are taken under the path measure induced by the policy currently under discussion.*

We next derive the little $q$-function $q(t,\boldsymbol{x},\boldsymbol{u};\pi)$, which quantifies the instantaneous advantage of taking action $\boldsymbol{u}$ at $(t,\boldsymbol{x})$ and then reverting to the current policy $\pi$. We sketch the main idea and defer the details to supplementary materials Section S.1.1.

Fix $\delta_t>0$, $\boldsymbol{u}\in\mathcal{A}$, and a baseline policy $\pi$. We consider a perturbed control: on the short interval $[t,t+\delta_t)$, we apply the constant action $\boldsymbol{u}$, and for $s\geq t+\delta_t$, we follow $\pi$. Let $(\boldsymbol{X}_s^{\boldsymbol{u}})_{s\geq t}$ denote the resulting state process with $\boldsymbol{X}_t^{\boldsymbol{u}}=\boldsymbol{x}$ (i.e., it solves the strict-control SDE (2.1) on $[t,t+\delta_t)$ with action $\boldsymbol{u}$ and then the exploratory SDE (2.2) under $\pi$ on $[t+\delta_t,\infty)$, initialized at $(t+\delta_t,\boldsymbol{X}_{t+\delta_t}^{\boldsymbol{u}})$). Define the corresponding discounted reward $Q_{\delta_t}(t,\boldsymbol{x},\boldsymbol{u};\pi)$ by

$$Q_{\delta_t}(t,\boldsymbol{x},\boldsymbol{u};\pi):=\mathbb{E}\Big[\int_t^{t+\delta_t}e^{-\beta(s-t)}\,f\big(s,\boldsymbol{X}_s^{\boldsymbol{u}},\boldsymbol{u}\big)\,\mathrm{d}s+\int_{t+\delta_t}^\infty e^{-\beta(s-t)}\,\tilde{f}\big(s,\boldsymbol{X}_s^{\boldsymbol{u}};\pi\big)\,\mathrm{d}s\mid\boldsymbol{X}_t^{\boldsymbol{u}}=\boldsymbol{x}\Big], \tag{3.6}$$

A first-order expansion (see Section S.1.1 in the supplementary materials) yields

$$\begin{aligned}Q_{\delta_t}\big(t,\boldsymbol{x},\boldsymbol{u};\pi\big)=J\big(t,\boldsymbol{x};\pi\big)+\Big(&\partial_tJ\big(t,\boldsymbol{x};\pi\big)\\&+\mathscr{H}\big(t,\boldsymbol{x},\boldsymbol{u},J(t,\boldsymbol{x};\pi),\nabla_xJ\big(t,\boldsymbol{x};\pi\big),\nabla_x^2J\big(t,\boldsymbol{x};\pi\big)\big)-\beta J\big(t,\boldsymbol{x};\pi\big)\Big)\,\delta_t+o(\delta_t),\end{aligned} \tag{3.7}$$

where $\mathscr{H}$ is the Hamiltonian defined in (2.11). This motivates the definition of the (little) $q$-function:

$$q(t,\boldsymbol{x},\boldsymbol{u};\pi):=\partial_tJ(t,\boldsymbol{x};\pi)+\mathscr{H}\big(t,\boldsymbol{x},\boldsymbol{u},J(t,\boldsymbol{x};\pi),\nabla_xJ(t,\boldsymbol{x};\pi),\nabla_x^2J(t,\boldsymbol{x};\pi)\big)-\beta J(t,\boldsymbol{x};\pi). \tag{3.8}$$

Indeed, $q(t,\boldsymbol{x},\boldsymbol{u};\pi)$ is the leading-order marginal gain per unit time of deviating from $\pi$ to $\boldsymbol{u}$.

Compared with [42], our definition includes the additional time-derivative term $\frac{\partial J}{\partial t}$ arising from time inhomogeneity. Compared with [26], we incorporate jump-diffusion dynamics and an infinite-horizon discounted objective through the Hamiltonian. Closely related jump extensions of the little-$q$ framework include [9, 18]; [9] focuses on Poisson point processes with Tsallis entropy regularization, while [18] considers a different use of the $q$-function for policy updates.

**3.2.2. Policy Gradient.** With the time–inhomogeneous $q$-function introduced, we are now ready to derive a policy gradient formula in our infinite-horizon jump-diffusion setting. We begin with two lemmas.

LEMMA 3.2. *Under the conditions in Definition 3.1, for any measurable function* $\varphi : [0,\infty) \times \mathbb{R}^d \to \mathbb{R}$ *such that* $\mathbb{E}\big[\int_t^\infty e^{-\beta(s-t)}|\varphi(s, \boldsymbol{X}_s^\pi)|\,\mathrm{d}s\big] < \infty$, *we have*

$$\mathbb{E}\Big[\int_t^\infty e^{-\beta(s-t)}\,\varphi(s, \boldsymbol{X}_s^\pi)\,\mathrm{d}s\Big] = \int_{[t,\infty)\times\mathbb{R}^d} \varphi(s,\boldsymbol{y})\,\mu^{\pi,t,\boldsymbol{x}}(\mathrm{d}s,\mathrm{d}\boldsymbol{y})\,. \tag{3.9}$$

This lemma extends [42, Lemma 1] to our time-inhomogeneous setting and follows from the occupation time formula. It states that any discounted pathwise reward $\varphi(s, \boldsymbol{X}_s^\pi)$ can be expressed as an integral of $\varphi$ with respect to the discounted time-state occupation measure $\mu^{\pi,t,\boldsymbol{x}}$. This identity will allow us to pass between expectations along trajectories and integrals over time-state space, which is crucial for writing performance differences in a concise integral form.

LEMMA 3.3. *Let* $\pi$ *and* $(\boldsymbol{X}_s^\pi)_{s\geq t}$ *be as in Definition* 3.1, *and let* $\varphi \in C^{1,2}([0,\infty)\times\mathbb{R}^d)$. *Assume that* $\mathbb{E}\int_t^\infty e^{-\beta(s-t)}\big(|\partial_s\varphi| + |\mathcal{L}^\pi\varphi| + \beta|\varphi|\big)(s, \boldsymbol{X}_s^\pi)\,\mathrm{d}s < \infty$, *and that* $\lim_{T\to\infty}\mathbb{E}\big[e^{-\beta(T-t)}|\varphi(T, \boldsymbol{X}_T^\pi)|\big] = 0$. *Then, for all* $t \geq 0$ *and* $\boldsymbol{x} \in \mathbb{R}^d$,

$$\mathbb{E}\bigg[\int_t^\infty e^{-\beta(s-t)}\big(-\partial_s\varphi - \mathcal{L}^\pi\varphi + \beta\varphi\big)(s, \boldsymbol{X}_s^\pi)\,\mathrm{d}s\bigg] = \varphi(t,\boldsymbol{x}), \tag{3.10}$$

*where* $\mathcal{L}^\pi$ *is defined in* (2.7).

The proof follows the same argument as [42, Lemma 8], with the addition of $\partial_s\varphi$ to account for the time inhomogeneity and a modified generator $\mathcal{L}^\pi$ to incorporate the jump terms. For brevity, the detailed proof is omitted. The discounted integrability and transversality conditions allow functions of polynomial growth, provided the state process has sufficiently high moments whose growth rate is dominated by $\beta$.

Since Lemma 3.3 holds for any $\varphi \in C^{1,2}$ with certain property, we may replace $\varphi$ by $J(\cdot,\cdot;\hat{\pi})$ that depends on a different stochastic control $\hat{\pi}$, whenever it is regular enough. Therefore, quantities defined under $\hat{\pi}$ can be represented using the generator $\mathcal{L}^\pi$ while expectations are taken along trajectories induced by $\pi$. This device is used below to compare $J(\cdot;\pi)$ and $J(\cdot;\hat{\pi})$.

THEOREM 3.4 (Policy gradient). *Let* $\pi$ *and* $\hat{\pi}$ *be two stochastic policies satisfying the conditions of Definition 3.1, and let* $\mu^{\hat{\pi},t,\boldsymbol{x}}$ *be the discounted occupation measure induced by* $\hat{\pi}$ *starting from* $(t,\boldsymbol{x})$. *Let* $J(t,\boldsymbol{x};\pi)$ *be the value function under* $\pi$. *Assume that* $J(\cdot,\cdot;\pi) \in C^{1,2}([0,\infty)\times\mathbb{R}^d)$ *is a classical solution of the PIDE* (2.6), *and satisfies the conditions of Lemma 3.3 along the state process* $\boldsymbol{X}^{\hat{\pi}}$. *Let* $q(t,\boldsymbol{x},\boldsymbol{u};\pi)$ *be the corresponding time-inhomogeneous q-function defined in* (3.8). *Then*

$$J(t,\boldsymbol{x};\hat{\pi}) - J(t,\boldsymbol{x};\pi) = \tfrac{1}{\beta}\,\mathbb{E}_{(s,\boldsymbol{X}_s^{\hat{\pi}})\sim\beta\mu^{\hat{\pi},t,\boldsymbol{x}},\,\boldsymbol{u}\sim\hat{\pi}(\cdot|s,\boldsymbol{X}_s^{\hat{\pi}})}\Big[q(s,\boldsymbol{X}_s^{\hat{\pi}},\boldsymbol{u};\pi) - \gamma\log\hat{\pi}(\boldsymbol{u}\mid s,\boldsymbol{X}_s^{\hat{\pi}})\Big]. \tag{3.11}$$

*Now let* $\{\pi_\theta(\boldsymbol{u}\mid t,\boldsymbol{x})\}_{\theta\in\Theta}$ *be a family of parameterized stochastic policies, and fix* $\theta_0 \in \Theta$. *For each* $\theta$, *let* $\mu^{\theta,t,\boldsymbol{x}}$ *denote the discounted occupation measure of* $(\boldsymbol{X}_s^{\pi_\theta})_{s\geq t}$ *under* $\pi_\theta$ *with* $\boldsymbol{X}_t = \boldsymbol{x}$, *and let* $q(\cdot;\pi_\theta)$ *be the associated q-function. Assume additionally that all* $\pi_\theta$ *in a neighborhood* $U$ *of* $\theta_0$ *are admissible and have a common support,* $\theta \mapsto \pi_\theta(\boldsymbol{u}\mid t,\boldsymbol{x})$ *is differentiable. Assume further that the domination and weighted continuity conditions stated in Section S.1.2 of the supplementary materials hold. Then, the mapping* $\theta \mapsto J(t,\boldsymbol{x};\pi_\theta)$ *is Gâteaux differentiable at* $\theta_0$, *and given by*

$$\nabla_\theta J(t,\boldsymbol{x};\pi_\theta)\big|_{\theta=\theta_0} = \tfrac{1}{\beta}\,\mathbb{E}_{(s,\boldsymbol{y})\sim\beta\mu^{\theta_0,t,\boldsymbol{x}},\,\boldsymbol{u}\sim\pi_{\theta_0}(\cdot|s,\boldsymbol{y})}\Big[\nabla_\theta\log\pi_\theta(\boldsymbol{u}\mid s,\boldsymbol{y})\big|_{\theta=\theta_0}\,A_{\mathrm{ent}}(s,\boldsymbol{y},\boldsymbol{u};\theta_0)\Big], \tag{3.12}$$

*where the exploratory advantage is defined by*

$$A_{\mathrm{ent}}(s,\boldsymbol{y},\boldsymbol{u};\theta_0) := q(s,\boldsymbol{y},\boldsymbol{u};\pi_{\theta_0}) - \gamma \log \pi_{\theta_0}(\boldsymbol{u} \mid s,\boldsymbol{y}). \tag{3.13}$$

The proof of the policy-gradient formula (3.12) is given in Section S.1.2 in the supplementary materials. This argument extends [42, Theorem 3]. In particular, when $\gamma = 0$, (3.12) reduces to the classical policy-gradient formula [30, 36].

Once Theorem 3.4 is established, we obtain an explicit representation of the policy gradient and can, in principle, learn the optimal policy via reinforcement learning. However, the $q$-function is primarily a formal object: evaluating it may require derivatives such as $\nabla_{\boldsymbol{x}} J$ and $\nabla^2_{\boldsymbol{x}} J$, which is computationally expensive and numerically delicate. Moreover, in model-free settings the SDE coefficients $(\boldsymbol{b},\boldsymbol{\sigma},\boldsymbol{\alpha})$ are unknown, so the Hamiltonian terms cannot be computed directly. Therefore, practical implementations must rely on tractable approximations of $q$. Two main approaches have been explored. First, [9, 26] approximate $q$ by neural networks and learn it via martingale properties, then update the policy via the Gibbs form implied by the $q$-function. Second, [42] relates $q$-function with $J$ without requiring derivatives, yielding an estimator akin to the generalized advantage estimation (GAE) [38]. Our approach follows the latter and leads to the next result under our setting with proof presented in Section S.1.3 in the supplementary materials.

LEMMA 3.5 (Approximation of $q$-function). *Fix $\beta > 0$ and a stochastic policy $\pi$. Let $J(\cdot,\cdot;\pi)$ denote the corresponding discounted reward. Assume $J \in C^{1,2}([0,\infty)\times\mathbb{R}^d)$ and that there exists $\delta_0 > 0$ such that, for every $t \geq 0$ and every $\delta_t \in (0,\delta_0]$, $\mathbb{E}\Big[\sup_{s\in[t,t+\delta_t)} \big(|J(s,\boldsymbol{X}^{\boldsymbol{u}}_s;\pi)| + |\frac{\partial J}{\partial t}(s,\boldsymbol{X}^{\boldsymbol{u}}_s;\pi)| + |\nabla_x J(s,\boldsymbol{X}^{\boldsymbol{u}}_s;\pi)| + \|\nabla^2_x J(s,\boldsymbol{X}^{\boldsymbol{u}}_s;\pi)\|\big) \,\Big|\, \mathcal{F}_t\Big] < \infty$, where $(\boldsymbol{X}^{\boldsymbol{u}}_s)_{s\geq t}$ is defined before (3.6) with $\boldsymbol{X}^{\boldsymbol{u}}_t = \boldsymbol{x}$. Define the quantity*

$$\tilde{q}_{\delta_t}(t,\boldsymbol{X}^{\boldsymbol{u}}_t,\boldsymbol{u};\pi) := \frac{1}{\delta_t}\big[f(t,\boldsymbol{X}^{\boldsymbol{u}}_t,\boldsymbol{u})\,\delta_t + e^{-\beta\delta_t}\, J\big(t+\delta_t, \boldsymbol{X}^{\boldsymbol{u}}_{t+\delta_t};\pi\big) - J\big(t,\boldsymbol{X}^{\boldsymbol{u}}_t;\pi\big)\big]. \tag{3.14}$$

*Then, as $\delta_t \to 0$,*

$$\mathbb{E}\big[\,\tilde{q}_{\delta_t}(t,\boldsymbol{x},\boldsymbol{u};\pi)\,\big|\,\mathcal{F}_t\big] = q\big(t,\boldsymbol{x},\boldsymbol{u};\pi\big) + o(1). \tag{3.15}$$

*Therefore, $\tilde{q}_{\delta_t}(t,\boldsymbol{x},\boldsymbol{u};\pi)$ is a first-order asymptotically unbiased estimator of the $q$-function.*

*Remark* 3.6. A convenient sufficient condition for the local integrability assumption in Lemma 3.5 is that $J$, $\partial_t J$, $\nabla_x J$, and $\nabla^2_x J$ have polynomial growth in $\boldsymbol{x}$, and that $\boldsymbol{X}^{\boldsymbol{u}}$ satisfies moment estimates of sufficiently high order on $[t,t+\delta_0]$. Such moment estimates follow under standard Lipschitz, growth, and jump-integrability conditions on the state coefficients; see, e.g., [18, Proposition 2] and [31, Theorem 1.19].

Therefore, when updating the actor using (3.12)–(3.13), we replace $q$ by $\tilde{q}_{\delta_t}$, and correspondingly approximate $A_{\mathrm{ent}}(s,\boldsymbol{y},\boldsymbol{u};\theta_0)$ by $\tilde{q}_{\delta_t}(s,\boldsymbol{y},\boldsymbol{u};\pi_{\theta_0}) - \gamma\log\pi_{\theta_0}(\boldsymbol{u}\mid s,\boldsymbol{y})$.

**3.3. The Actor-Critic Scheme.** We describe the proposed online actor-critic scheme for time-inhomogeneous jump diffusion control problems. The new scheme admits both model-free and model-based implementations. The actor update and the standard TD-based critic update require only sampled states, actions, rewards, and next states. The optional martingale-corrected critic update additionally uses the diffusion and jump coefficients, together with the corresponding observed or simulated noise increments. In our numerical experiments below, we use known simulators in

order to compare the learned policies and value functions with analytical or high-accuracy benchmarks.

Fix a deterministic step size $\delta_t$ and consider the uniform grid $0 = t_0 < t_1 < \cdots < t_K$ with $t_k = k\,\delta_t$. We parameterize the actor and critic by neural networks, denoted by $\pi_\theta(\boldsymbol{u} \mid t, \boldsymbol{x})$ and $V_\psi(t, \boldsymbol{x})$, and update $(\theta, \psi)$ iteratively via policy gradient and policy evaluation. At each iteration, a minibatch of $L$ trajectories is used, which we denote by $\{\boldsymbol{X}_k^{(\ell)}\}_{k=0}^K$, $\ell = 1, \ldots, L$. Specifically, at time $t_k$, we sample an action $\boldsymbol{u}_k^{(\ell)}$ from the current policy $\pi_\theta(\cdot \mid t_k, \boldsymbol{X}_k^{(\ell)})$, apply the sampled action and observe the next state $\boldsymbol{X}_{k+1}^{(\ell)}$ from the environment. When a simulator is available, the transition may instead be generated using an Euler discretization of (2.1) with $\boldsymbol{u}_k^{(\ell)}$, see details in Section S.3.1 of the supplementary materials. The discounted reward $f_{k,\ell}$ accumulated over $[t_k, t_{k+1}]$ is also observed from the environment or approximated by $f_{k,\ell} := f(t_k, \boldsymbol{X}_k^{(\ell)}, \boldsymbol{u}_k^{(\ell)})\,\delta_t$ when the reward function $f$ is known.

**Critic.** To update the critic parameters $\psi$ in $V_\psi(t, \boldsymbol{x})$, we construct the TD error (3.2) (or the martingale-corrected TD error (3.4)) along sampled trajectories using $\bar{V}_{\bar{\psi}}$ for the bootstrapped next-state target. The target network $\bar{V}_{\bar{\psi}}$ decouples the bootstrapping target from the online critic being optimized, thereby reducing target drift and improving stability during training, following the target-network idea used in soft actor-critic (SAC) [20]. Specifically, for trajectory $\ell$ at time $t_k$, define

$$\delta_{\mathrm{TD},k}^{(\ell)} := f_{k,\ell} - \gamma \log \pi_\theta^{(\ell)}(t_k)\delta_t + e^{-\beta\delta_t}\,\bar{V}_{\bar{\psi}}\big(t_{k+1}, \boldsymbol{X}_{k+1}^{(\ell)}\big) - V_\psi\big(t_k, \boldsymbol{X}_k^{(\ell)}\big), \tag{3.16}$$

$$\tilde{\delta}_{\mathrm{TD},k}^{(\ell)} := \delta_{\mathrm{TD},k}^{(\ell)} - \big(\nabla_x V_\psi\big(t_k, \boldsymbol{X}_k^{(\ell)}\big)^\top \boldsymbol{\sigma}\big(t_k, \boldsymbol{X}_k^{(\ell)}, \boldsymbol{u}_k^{(\ell)}\big)\big)\Delta \boldsymbol{W}_k^{(\ell)} - \Big( \sum_{i=N_{t_k}^{(\ell)}+1}^{N_{t_{k+1}}^{(\ell)}} \big[V_\psi\big(t_k, \boldsymbol{X}_k^{(\ell)} + \boldsymbol{\alpha}(t_k, \boldsymbol{X}_k^{(\ell)}, \boldsymbol{u}_k^{(\ell)}, \boldsymbol{z}_i^{(\ell)})\big) - V_\psi\big(t_k, \boldsymbol{X}_k^{(\ell)}\big)\big] - \delta_t V_{\mathrm{non}}\big(t_k, \boldsymbol{X}_k^{(\ell)}, \boldsymbol{u}_k^{(\ell)}\big)\Big), \tag{3.17}$$

where $\Delta \boldsymbol{W}_k^{(\ell)} := \boldsymbol{W}_{t_{k+1}}^{(\ell)} - \boldsymbol{W}_{t_k}^{(\ell)} \sim \mathcal{N}(0, \delta_t \boldsymbol{I}_d)$, $N_{t_k}^{(\ell)}$ counts the number of jumps on the trajectory $\ell$, $\{\boldsymbol{z}_i^{(\ell)}\}_{i \geq 1}$ are the corresponding jump sizes. The quantity $V_{\mathrm{non}}(t, \boldsymbol{x}, \boldsymbol{u}) := \int_{\mathbb{R}^d} \big[V_\psi\big(t, \boldsymbol{x} + \boldsymbol{\alpha}(t, \boldsymbol{x}, \boldsymbol{u}, \boldsymbol{z})\big) - V_\psi(t, \boldsymbol{x})\big]\nu(\mathrm{d}\boldsymbol{z})$ is the instantaneous compensator rate of the jump increments of $V_\psi$. Indeed, since $\widetilde{N}(\mathrm{d}t, \mathrm{d}\boldsymbol{z}) = N(\mathrm{d}t, \mathrm{d}\boldsymbol{z}) - \nu(\mathrm{d}\boldsymbol{z})\mathrm{d}t$, the jump martingale increment over $[t_k, t_{k+1}]$ is approximated by the realized jump sum minus $\delta t\, V_{\mathrm{non}}(t_k, \boldsymbol{X}_k, \boldsymbol{u}_k)$. Under Poisson jumps, $\nu(\mathrm{d}\boldsymbol{z}) = \sum_{i=1}^d \lambda_i \delta_{\boldsymbol{e}_i}(\mathrm{d}\boldsymbol{z})$, the integral is evaluated exactly as $V_{\mathrm{non}}(t, \boldsymbol{x}, \boldsymbol{u}) = \sum_{i=1}^d \lambda_i \big[V_\psi\big(t, \boldsymbol{x} + \boldsymbol{\alpha}(t, \boldsymbol{x}, \boldsymbol{u}, \boldsymbol{e}_i)\big) - V_\psi(t, \boldsymbol{x})\big]$. For a general known finite-activity Lévy measure, approximation and implementation details are provided in Section S.3.2 (supplementary materials). Note that this term is required only for the martingale-corrected TD update.

Given a minibatch of $L$ trajectories and $K_{\mathrm{critic}}$ time steps, we update $\psi$ by minimizing the empirical mean-squared (martingale-corrected) TD error

$$L_{\mathrm{critic}}(\psi) = \frac{1}{LK_{\mathrm{critic}}} \sum_{\ell=1}^{L} \sum_{k=0}^{K_{\mathrm{critic}}-1} (\delta_{\mathrm{TD},k}^{(\ell)})^2 \ \text{ or } \ \frac{1}{LK_{\mathrm{critic}}} \sum_{\ell=1}^{L} \sum_{k=0}^{K_{\mathrm{critic}}-1} (\tilde{\delta}_{\mathrm{TD},k}^{(\ell)})^2. \tag{3.18}$$

When minimizing (3.18) with the usual one-step TD error, we apply semi-gradient: the bootstrapped target $\bar{V}_{\bar{\psi}}\big(t_{k+1}, \boldsymbol{X}_{k+1}^{(\ell)}\big)$ is held fixed when differentiating with respect to $\psi$, which is consistent with the martingale orthogonality condition in [24, 25]. The target critic parameters are then updated by Polyak averaging $\bar{\psi} \leftarrow \rho_c\,\bar{\psi} + (1 - \rho_c)\,\psi$, $\rho_c \in (0, 1)$, where $\rho_c$ is a prescribed averaging weight [20]. The martingale

correction is an optional variance-reduction device for the critic update and requires additional information about the model coefficients and noise increments. To isolate its variance-reduction effect, the supplementary materials Section S.3.3 compares the usual and martingale-corrected TD errors under the same numerical setting. This comparison demonstrates the effect of removing the martingale fluctuation rather than to provide a model-free performance comparison, since the martingale-corrected TD error uses additional information. The two supplementary comparisons indeed confirm that the martingale correction substantially reduces the TD-residual variance and that this variance reduction translates into improved value estimation and policy learning.

**Actor.** We parameterize the stochastic policy $\pi_\theta$ as a conditional normalizing flow [10, 32]. Specifically, the policy is defined as the pushforward of a Gaussian distribution $\mathcal{N}_\theta$[1]

$$\boldsymbol{z}_0 \sim \mathcal{N}_\theta(\cdot \mid t, \boldsymbol{x}) = \mathcal{N}\big(\bar{\boldsymbol{\mu}}_\theta(t, \boldsymbol{x}),\, \mathrm{Std}_\theta^2(t, \boldsymbol{x})\big), \tag{3.19}$$

through a learnable invertible normalizing flow $F_\theta(\cdot; t, \boldsymbol{x})$, followed by an optional differentiable squashing map $S$ (e.g., a `sigmoid` or `tanh`) that enforces control constraints when needed. The log-density of the resulting control sample is computed exactly by the change-of-variables formula,

$$\begin{aligned} \log \pi_\theta(\boldsymbol{u} \mid t, \boldsymbol{x}) = \log p_{\mathcal{N}_\theta}\big(\boldsymbol{z}_0 \mid \bar{\boldsymbol{\mu}}_\theta(t, \boldsymbol{x}), \mathrm{Std}_\theta(t, \boldsymbol{x})\big) - \log\big|\det J_F(\boldsymbol{z}_0; t, \boldsymbol{x})\big| \\ - \log\big|\det J_S(F_\theta(\boldsymbol{z}_0))\big|, \end{aligned} \tag{3.20}$$

where $J_F$ and $J_S$ denote the Jacobians of the flow $F$ and the squashing map $S$, respectively. This construction defines a flexible stochastic policy class that includes Gaussian policies as a special case[2], while retaining exact likelihood evaluation required for entropy regularization and policy-gradient optimization. The simultaneous parameterization of the Gaussian base distribution and the flow map is generally non-identifiable. This does not affect the method, since only the induced policy distribution is relevant. A fixed standard Gaussian base could alternatively be used; we retain a learnable Gaussian base because it includes learnable Gaussian policies when the flow is the identity.

Our implementation of $F_\theta$ uses rational-quadratic spline coupling layers [17]. Their coupling structure yields a triangular Jacobian with respect to the latent input, so $\log|\det J_F|$ is evaluated as a sum of one-dimensional spline derivatives rather than by forming a dense Jacobian matrix. Gradients with respect to $\theta$ are computed by automatic differentiation through the resulting scalar log-density. Further implementation details are provided in Appendix A. When constraints are imposed, the present implementation considers box-type constraints, for which $S$ is chosen as a coordinatewise `sigmoid` or `tanh` map. Its Jacobian is diagonal, so $\log|\det J_S|$ is computed as a sum of coordinatewise log-derivatives. For more general constraint sets, a problem-specific invertible reparameterization may be required.

[1]Even in the unregularized case ($\gamma = 0$), where the optimal control is deterministic, we still adopt a Gaussian policy parameterization. This choice is supported by [30, 39], which establish the convergence of policy-gradient methods with stochastic policies and show that, in terms of solvability, they are equivalent to the corresponding standard control problem.

[2]When the Hamiltonian $\mathscr{H}$ is quadratic in $\boldsymbol{u}$, the optimal randomized policy $\pi^*(\boldsymbol{u} \mid t, \boldsymbol{x}) \propto \exp(\mathscr{H}(t, \boldsymbol{x}, \boldsymbol{u})/\gamma)$ is Gaussian; if the control domain is unconstrained, we may also omit the squashing map, so that only the first term in (3.20) remains.

In implementation, samples of $\pi_\theta(\cdot \mid t_k, \boldsymbol{X}_k^{(\ell)})$ are generated using

$$\begin{aligned} \boldsymbol{z}_{0,k}^{(\ell)} &= \bar{\boldsymbol{\mu}}_\theta\big(t_k, \boldsymbol{X}_k^{(\ell)}\big) + \mathrm{Std}_\theta\big(t_k, \boldsymbol{X}_k^{(\ell)}\big) \odot \boldsymbol{\varepsilon}_k^{(\ell)},\ \boldsymbol{\varepsilon}_k^{(\ell)} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_m), \\ \boldsymbol{z}_{F,k}^{(\ell)} &= F_\theta\big(\boldsymbol{z}_{0,k}^{(\ell)}; t_k, \boldsymbol{X}_k^{(\ell)}\big), \quad \boldsymbol{u}_k^{(\ell)} = S(\boldsymbol{z}_{F,k}^{(\ell)}) \in [\boldsymbol{u}_{\min}, \boldsymbol{u}_{\max}]^m. \end{aligned} \tag{3.21}$$

To update the actor parameters $\theta$, we form the one-step advantage estimator using (3.13) and Lemma 3.5:

$$\hat{A}_k^{(\ell)} := \frac{1}{\delta_t}\Big(f_{k,\ell} + e^{-\beta\delta_t} V_\psi\big(t_{k+1}, \boldsymbol{X}_{k+1}^{(\ell)}\big) - V_\psi\big(t_k, \boldsymbol{X}_k^{(\ell)}\big)\Big) - \gamma \log \pi_\theta^{(\ell)}(t_k), \tag{3.22}$$

where $\log \pi_\theta^{(\ell)}(t_k) := \log \pi_\theta(\boldsymbol{u}_k^{(\ell)} \mid t_k, \boldsymbol{X}_k^{(\ell)})$ using (3.20). Given a minibatch of $L$ trajectories and $K_{\text{actor}}$ time steps, we update $\theta$ by minimizing the policy-gradient surrogate

$$L_{\text{actor}}(\theta) = -\frac{1}{\beta L K_{\text{actor}}} \sum_{\ell=1}^{L} \sum_{k=0}^{K_{\text{actor}}-1} \log \pi_\theta^{(\ell)}(t_k)\, \mathrm{stopgrad}\big(\hat{A}_k^{(\ell)}\big), \tag{3.23}$$

where $\mathrm{stopgrad}(\cdot)$ indicates that $\hat{A}_k^{(\ell)}$ is treated as constant when differentiating with respect to $\theta$, consistent with Theorem 3.4.

Note that the one-step estimator in (3.22) may have large variance for small $\delta_t$, since the terms in parentheses contain Brownian and compensated-jump martingale increments of order $\sqrt{\delta_t}$ which become $\delta_t^{-1/2}$ after division by $\delta_t$. Thus we reduce this variance by averaging over $L$ trajectories and $K_{\text{actor}}$ time steps, cf. (3.23). When the model coefficients and the noise increments are available, one may further subtract the corresponding martingale correction from the actor estimator, analogously to the optional martingale-corrected TD error; i.e. subtracting $(\delta_{\text{TD},k}^{(\ell)} - \tilde{\delta}_{\text{TD},k}^{(\ell)})/\delta_t$ from $\hat{A}_k^{(\ell)}$.

Combining (3.18) and (3.23) yields the online time-inhomogeneous actor-critic procedure summarized in Alg. 3.1. We note that the underlying control problem is infinite-horizon: the finite sum over $k$ corresponds to an optimization window used for stability and variance reduction, and does not impose a finite terminal time.

**4. Numerical Experiments.** In this section, we illustrate the proposed actor-critic framework through a set of representative numerical examples. We consider three problems of increasing complexity: a linear-quadratic (LQ) control problem with jump diffusion (Section 4.1), the Merton portfolio optimization problem (Section 4.2), and a multi-agent portfolio game (Section 4.3). These examples are chosen to demonstrate the flexibility of the method across settings with known analytical structure, nonlinear dynamics, and strategic interactions, as well as to assess its empirical stability and performance in time-inhomogeneous jump-diffusion environments.

**Metrics.** We assess the learned actor and critic networks using trajectory-level metrics. For each experiment, we analyze the learned (i) state trajectory $\hat{\boldsymbol{X}}_t$, (ii) value function $\hat{V}(t, \hat{\boldsymbol{X}}_t)$, and (iii) control process (the feedback control $\hat{\boldsymbol{u}}(t, \hat{\boldsymbol{X}}_t)$ when $\gamma = 0$, or the mean of $\hat{\boldsymbol{u}}_t \sim \hat{\pi}(\cdot \mid t, \hat{\boldsymbol{X}}_t)$ in the exploratory case $\gamma > 0$).

When a benchmark solution $(\boldsymbol{u}^*, V, \boldsymbol{X})$ is available, we report time-averaged relative mean-square errors (RMSEs) on $[0, T_{\text{eval}}]$:

$$\mathcal{E}_X(T_{\text{eval}}) := \frac{\int_0^{T_{\text{eval}}} \|\hat{\boldsymbol{X}}_t - \boldsymbol{X}_t\|^2 \,\mathrm{d}t}{\int_0^{T_{\text{eval}}} \|\boldsymbol{X}_t\|^2 \,\mathrm{d}t + \varepsilon_X}, \qquad \mathcal{E}_V(T_{\text{eval}}) := \frac{\int_0^{T_{\text{eval}}} |\hat{V}(t, \hat{\boldsymbol{X}}_t) - V(t, \boldsymbol{X}_t)|^2 \,\mathrm{d}t}{\int_0^{T_{\text{eval}}} |V(t, \boldsymbol{X}_t)|^2 \,\mathrm{d}t + \varepsilon_V}. \tag{4.1}$$

**Algorithm 3.1** Time-inhomogeneous actor-critic for infinite-horizon jump-diffusion

**Require:** Step size $\delta_t$; discount $\beta$; entropy weight $\gamma$; number of time points $K$; update periods $K_{\text{critic}}, K_{\text{actor}}$; iterations $N_{\text{itr}}$; minibatch $L$; value net $V_\psi(t,\boldsymbol{x})$; target net $\bar{V}_{\bar{\psi}}(t,\boldsymbol{x})$; Polyak coefficient $\rho_c \in (0,1)$; stochastic policy $\pi_\theta(\cdot \mid t,\boldsymbol{x})$.

**for** it $= 1$ to $N_{\text{itr}}$ **do**
  Set initial time $t \leftarrow 0$ and initial states $\boldsymbol{X}_0 \leftarrow \boldsymbol{X}_{\text{init}}$.
  $V \leftarrow V_\psi(t, \boldsymbol{X})$.
  $\mathcal{L}_{\text{critic}} \leftarrow 0,\ n_{\text{critic}} \leftarrow 0; \quad \mathcal{L}_{\text{actor}} \leftarrow 0,\ n_{\text{actor}} \leftarrow 0.$
  **for** $k = 0$ to $K-1$ **do**
    Sample $\boldsymbol{u}_k^{(\ell)}$ according to (3.21).
    Calculate the log-density $\log \pi_\theta^{(\ell)}(t_k)$ via (3.20).
    Observe the next state $\boldsymbol{X}_{k+1}^{(\ell)}$ from the environment or evolve from $\boldsymbol{X}_k^{(\ell)}$ via Euler scheme.
    Compute the TD error $\delta_{\text{TD},k}^{(\ell)}$ using (3.16) or (3.17).
    $\mathcal{L}_{\text{critic}} \leftarrow \mathcal{L}_{\text{critic}} + \sum_{\ell=1}^{L} \|\delta_{\text{TD},k}^{(\ell)}\|_2^2; \quad n_{\text{critic}} \leftarrow n_{\text{critic}} + 1.$
    **if** $n_{\text{critic}} \bmod K_{\text{critic}} = 0$ **then**
      Update $\psi$ by one optimizer step on $\mathcal{L}_{\text{critic}}/LK_{\text{critic}}$. ▷ critic objective (3.18)
      $\bar{\psi} \leftarrow \rho_c\, \bar{\psi} + (1-\rho_c)\, \psi.$
      $\mathcal{L}_{\text{critic}} \leftarrow 0; \quad n_{\text{critic}} \leftarrow 0.$
    **end if**
    Compute the GAE estimator $\hat{A}_k^{(\ell)}$ according to (3.22)
    $\mathcal{L}_{\text{actor}} \leftarrow \mathcal{L}_{\text{actor}} - \beta^{-1} \sum_{\ell=1}^{L} \log \pi_\theta^{(\ell)}(t_k)\ \text{stopgrad}(\hat{A}_k^{(\ell)}); \quad n_{\text{actor}} \leftarrow n_{\text{actor}} + 1.$
    **if** $n_{\text{actor}} \bmod K_{\text{actor}} = 0$ **then**
      Update $\theta$ by one optimizer step on $\mathcal{L}_{\text{actor}}/LK_{\text{actor}}$. ▷ actor objective (3.23)
      $\mathcal{L}_{\text{actor}} \leftarrow 0; \quad n_{\text{actor}} \leftarrow 0.$
    **end if**
  **end for**
**end for**

For the learned control, we use the RMSE when $\gamma = 0$, and a distributional discrepancy when $\gamma > 0$:

$$\mathcal{E}_u(T_{\text{eval}}) := \begin{cases} \dfrac{\int_0^{T_{\text{eval}}} \|\hat{\boldsymbol{u}}(t, \hat{\boldsymbol{X}}_t) - \boldsymbol{u}^*(t, \boldsymbol{X}_t)\|^2 \,\mathrm{d}t}{\int_0^{T_{\text{eval}}} \|\boldsymbol{u}^*(t, \boldsymbol{X}_t)\|^2 \,\mathrm{d}t + \varepsilon_u}, & \gamma = 0, \\ \dfrac{1}{T_{\text{eval}}} \displaystyle\int_0^{T_{\text{eval}}} \mathrm{KL}\big(\pi^*(\cdot \mid t, \hat{\boldsymbol{X}}_t) \,\big\|\, \hat{\pi}(\cdot \mid t, \hat{\boldsymbol{X}}_t)\big)\,\mathrm{d}t, & \gamma > 0, \end{cases} \tag{4.2}$$

since for $\gamma > 0$, both the benchmark and learned controls are stochastic policies, denoted by $\pi^*(\cdot \mid t,\boldsymbol{x})$ and $\hat{\pi}(\cdot \mid t,\boldsymbol{x})$. The constants $\varepsilon_X, \varepsilon_V, \varepsilon_u > 0$ are small stabilizers included to avoid division by zero.

*Poisson jump specification.* In the experiments, we consider a discrete Lévy measure corresponding to Poisson jumps: $\nu(\mathrm{d}\boldsymbol{z}) = \sum_{i=1}^{d} \lambda_i\, \delta_{\boldsymbol{e}_i}(\mathrm{d}\boldsymbol{z})$, where $\lambda_i > 0$ and $\boldsymbol{e}_i$ is the $i$-th canonical basis vector in $\mathbb{R}^d$. Under this specification, the jump measure is represented by independent Poisson processes $N_t^{(i)}$ with rates $\lambda_i$, so that $N(\mathrm{d}t, \boldsymbol{e}_i) = \mathrm{d}N_t^{(i)}$. Let $M_t^{(i)} := N_t^{(i)} - \lambda_i t$ be the compensated Poisson process, namely $\mathrm{d}M_t^{(i)} = \mathrm{d}N_t^{(i)} - \lambda_i\,\mathrm{d}t$. Then the jump term in (2.1) becomes $\int_{\mathbb{R}^d} \boldsymbol{\alpha}\big(t, \boldsymbol{X}_{t-}^{\pi}, \boldsymbol{u}_t, \boldsymbol{z}\big)\, \tilde{N}(\mathrm{d}t, \mathrm{d}\boldsymbol{z}) =$

$\sum_{i=1}^d \boldsymbol{\alpha}\big(t, \boldsymbol{X}^\pi_{t-}, \boldsymbol{u}_t, \boldsymbol{e}_i\big)\,\mathrm{d}M_t^{(i)}$. Accordingly, the nonlocal integral term in the Hamiltonian (2.11) and the generator $\mathcal{L}^{\boldsymbol{u}}$ in (2.8) reduces to $\sum_{i=1}^d \lambda_i \Big[V\big(t, \boldsymbol{x}+\boldsymbol{\alpha}(t,\boldsymbol{x},\boldsymbol{u},\boldsymbol{e}_i)\big) - V(t,\boldsymbol{x}) - \boldsymbol{\alpha}(t,\boldsymbol{x},\boldsymbol{u},\boldsymbol{e}_i)\cdot\nabla_{\boldsymbol{x}} V(t,\boldsymbol{x})\Big]$.

In each of the following sections, we (i) specify the control or game formulation, (ii) give the analytical solution when available, or present how benchmark solutions are obtained, and (iii) present the numerical results. All implementation details and model parameters are provided in Appendix A. Some benchmark derivations are standard in the stochastic control literature, and are included in the supplementary materials for completeness. All experiments are implemented in `PyTorch` and run on an NVIDIA RTX `4090` GPU. The code is available upon request and will be made public upon publication.

**4.1. Linear-Quadratic Control with Jump Diffusions.** We first consider a $d$-dimensional state $\boldsymbol{X}_t \in \mathbb{R}^d$ and a $m$-dimensional control $\boldsymbol{u}_t \in \mathbb{R}^m$, governed by the controlled jump-diffusion

$$\mathrm{d}\boldsymbol{X}_t = \boldsymbol{B}(t)\,\boldsymbol{u}_t\,\mathrm{d}t + \boldsymbol{\Sigma}(t)\,\mathrm{d}\boldsymbol{W}_t + \sum_{i=1}^d \alpha_i(t)\,\boldsymbol{e}_i\,\mathrm{d}M_t^{(i)}, \quad t \geq 0, \tag{4.3}$$

where $\boldsymbol{B}(t) \in \mathbb{R}^{d\times m}$, $\boldsymbol{\Sigma}(t) \in \mathbb{R}^{d\times d}$, $\boldsymbol{\alpha}(t) := (\alpha_1(t),\ldots,\alpha_d(t))^\top \in \mathbb{R}^d$, and $\boldsymbol{e}_i$ is the $i$th canonical basis vector in $\mathbb{R}^d$. The running reward is quadratic $f(t,\boldsymbol{x},\boldsymbol{u}) = -(\boldsymbol{u}^\top \boldsymbol{R}(t)\,\boldsymbol{u} + \boldsymbol{x}^\top \boldsymbol{Q}(t)\,\boldsymbol{x})$, where $\boldsymbol{R}(t) \in \mathbb{S}^m$ and $\boldsymbol{Q}(t) \in \mathbb{S}^d$ are prescribed positive definite model parameters. Consequently, $\boldsymbol{R}(t)^{-1}$ is well defined.

Recall that the value function satisfies (2.12) for the standard control ($\gamma = 0$) and (2.10) under entropy-regularization, with the integral term replaced by $\sum_{i=1}^d \lambda_i(t)\big(\varphi(t,\boldsymbol{x}+\alpha_i(t)\boldsymbol{e}_i) - \varphi(t,\boldsymbol{x}) - \alpha_i(t)\,\partial_{x_i}\varphi(t,\boldsymbol{x})\big)$. Therefore, the optimal policy and value function satisfy

$$\pi^*(\boldsymbol{u}\mid t,\boldsymbol{x}) = \mathcal{N}\Big(\boldsymbol{R}(t)^{-1}\boldsymbol{B}(t)^\top \boldsymbol{H}(t)\boldsymbol{x},\ \frac{\gamma}{2}\boldsymbol{R}(t)^{-1}\Big),\ V(t,\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{H}(t)\boldsymbol{x} + g_\gamma(t)\,, \tag{4.4}$$

where $\boldsymbol{H}(t) \in \mathbb{S}^d$ and $g_\gamma(t) \in \mathbb{R}$ solve

$$\begin{aligned} \boldsymbol{H}'(t) &= \beta\boldsymbol{H}(t) + \boldsymbol{Q}(t) - \boldsymbol{H}(t)\boldsymbol{B}(t)\boldsymbol{R}(t)^{-1}\boldsymbol{B}(t)^\top \boldsymbol{H}(t), \\ g_\gamma'(t) &= \beta g_\gamma(t) - \mathrm{Tr}\big(\boldsymbol{\Sigma}(t)\boldsymbol{\Sigma}(t)^\top \boldsymbol{H}(t)\big) - \mathrm{Tr}\Big(\boldsymbol{\Lambda}(t)\,\mathrm{diag}(\boldsymbol{\alpha}(t))\boldsymbol{H}(t)\,\mathrm{diag}(\boldsymbol{\alpha}(t))\Big) - c_\gamma(t), \end{aligned} \tag{4.5}$$

with $c_\gamma(t) = \frac{\gamma}{2}\big(m\log(\pi\gamma) - \log\det\boldsymbol{R}(t)\big)$, $\boldsymbol{\Lambda}(t) := \mathrm{diag}(\lambda_i(t))$. The proof for analytical solution, together with the boundary conditions for the ODE system (4.5), are presented in Section S.2.1 (supplementary materials). In the classical case $\gamma = 0$, by standard derivation, the optimal stochastic policy degenerates to the feedback control

$$\boldsymbol{u}^*(t,\boldsymbol{x}) = \boldsymbol{R}(t)^{-1}\boldsymbol{B}(t)^\top \boldsymbol{H}(t)\boldsymbol{x}. \tag{4.6}$$

**4.1.1. Time-Homogeneous Case.** We start with the time-homogeneous case, i.e., $\boldsymbol{B}(t) \equiv \boldsymbol{B}$, $\boldsymbol{\Sigma}(t) \equiv \boldsymbol{\Sigma}$, $\boldsymbol{\alpha}(t) \equiv \boldsymbol{\alpha}$, $\boldsymbol{\Lambda}(t) \equiv \boldsymbol{\Lambda}$, $\boldsymbol{R}(t) \equiv \boldsymbol{R}$ and $\boldsymbol{Q}(t) \equiv \boldsymbol{Q}$. In this case, there exists a stationary pair $(\boldsymbol{H}, g_\gamma)$ that solves:

$$\begin{aligned} \boldsymbol{0} &= \beta\boldsymbol{H} + \boldsymbol{Q} - \boldsymbol{H}\boldsymbol{B}\boldsymbol{R}^{-1}\boldsymbol{B}^\top \boldsymbol{H}, \\ \beta g_\gamma &= \mathrm{Tr}\big(\boldsymbol{\Sigma}\boldsymbol{\Sigma}^\top \boldsymbol{H}\big) + \mathrm{Tr}\Big(\boldsymbol{\Lambda}\ \mathrm{diag}(\boldsymbol{\alpha})\,\boldsymbol{H}\ \mathrm{diag}(\boldsymbol{\alpha})\Big) + \frac{\gamma}{2}\big(m\log(\pi\gamma) - \log\det\boldsymbol{R}\big). \end{aligned} \tag{4.7}$$

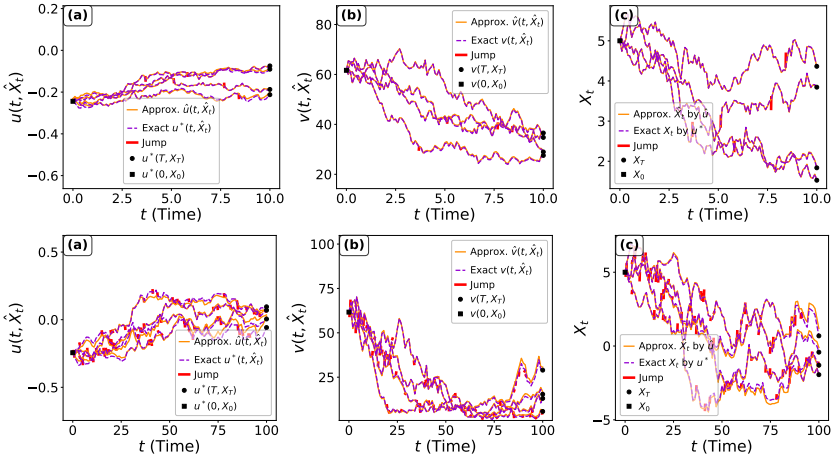



Fig. 1: Predicted (a) optimal control, (b) value function, and (c) state trajectory for the standard LQ problem with $d = 5$ on horizon $T = 10$ (top) and $T = 100$ (bottom). Parameters: $\gamma = 0$, $\boldsymbol{B} = 0.5\boldsymbol{I}_d$, $\boldsymbol{\Sigma} = 0.3\boldsymbol{I}_d$, $\boldsymbol{R} = 5\boldsymbol{I}_d$, $\boldsymbol{Q} = 0.5\boldsymbol{I}_d$, $\lambda_i = 0.2 + \frac{i-1}{d-1}(0.3 - 0.2)$ and $\alpha_i = 0.3 - \frac{i-1}{d-1}(0.3 - 0.2)$ for $i = 1, \ldots, d$.

The optimal stochastic policy remains Gaussian with mean $\boldsymbol{R}^{-1}\boldsymbol{B}^\top \boldsymbol{H}\boldsymbol{x}$ and covariance $\frac{\gamma}{2}\boldsymbol{R}^{-1}$, and the standard case follows by setting $c_\gamma = 0$.

**Standard LQ problem ($\gamma = 0$).** We first solve a 5-dimensional LQ control problem, and train the actor and critic networks $(\pi_\theta, V_\psi)$ using Algorithm 3.1 for $N_{\text{itr}} = 1{,}000$ iterations, with horizons $T \in \{10, 100\}$. We then freeze all networks and generate the approximated state trajectories $\hat{\boldsymbol{X}}_t$, value function $\hat{V}$ and the feedback control $\hat{\boldsymbol{u}}$ using step size $\delta_t = 0.01$. Figure 1 shows that the learned values are consistent with the analytical solution, and remain numerically stable even over the long horizon $T = 100$.

**Entropy-regularized LQ problem ($\gamma > 0$).** We next consider the entropy-regularized variant with $\gamma = 0.05$ under the same problem setup and evaluation procedure. For the LQ problem, the Hamiltonian $\mathscr{H}$ is quadratic in $\boldsymbol{u}$, so the optimal entropy-regularized policy is Gaussian. Accordingly, when parameterizing the actor we may treat the flow map as the identity and use only the Gaussian policy in (3.19). Equivalently, sampling actions reduces to setting $\boldsymbol{u}$ to the base variable in (3.21), i.e., $\boldsymbol{u} = \boldsymbol{z}_0$. Figure 2 reports the mean of stochastic control as well as the approximated state and value trajectories, showing that Alg. 3.1 remains numerically stable and accurate under exploration.

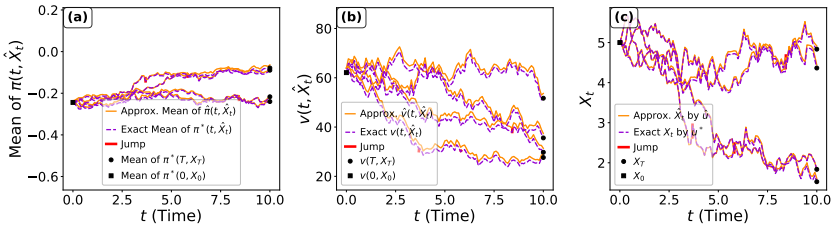



Fig. 2: Predicted (a) mean of stochastic policy, (b) value function, and (c) state trajectory for the entropy-regularized LQ problem with $d = 5$ and $\gamma = 0.05$. All remaining parameters and the evaluation protocol are the same as in Fig. 1, with horizon $T = 10$.

To test scalability in the state dimension, we repeat the entropy-regularized experiment ($\gamma = 0.05$) for $d \in \{1, 5, 20, 50\}$. Table 2 reports the RMSE for the value and control over three seeds. The value error $\mathcal{E}_V$ stays small across dimensions, whereas the control error $\mathcal{E}_u$ increases linearly with $d$, consistent with the greater difficulty of learning high-dimensional feedback under exploration noise.

Table 2: Training errors for the entropy-regularized ($\gamma = 0.05$) LQ problem across state dimensions $d$. All runs use $N_{\text{itr}} = 1{,}000$ iterations and results are averaged over three random seeds.

| | $d=1$ | $d=5$ | $d=20$ | $d=50$ |
|---|---|---|---|---|
| $\mathcal{E}_u$ | 0.1441 | 0.3992 | 1.9205 | 4.9366 |
| $\mathcal{E}_V$ | 0.0034 | 0.0036 | 0.0046 | 0.0038 |

**4.1.2. Time-Inhomogeneous Case. Convergent coefficients.** We next consider a time-inhomogeneous LQ problem such that as $t \to \infty$,

$$\boldsymbol{B}(t) \to \boldsymbol{B}_\infty,\ \boldsymbol{\Sigma}(t) \to \boldsymbol{\Sigma}_\infty,\ \boldsymbol{\alpha}(t) \to \boldsymbol{\alpha}_\infty,\ \boldsymbol{\Lambda}(t) \to \boldsymbol{\Lambda}_\infty,\ \boldsymbol{Q}(t) \to \boldsymbol{Q}_\infty,\ \boldsymbol{R}(t) \to \boldsymbol{R}_\infty,$$

with sufficiently fast convergence so that the discounted reward is well defined.

The computation of the benchmark solution is less direct than in the time-homogeneous case, since the ODE system (4.5) does not come with an explicit terminal boundary condition. For the present choice of convergent coefficients, the terminal boundary can be approximated by introducing a sufficiently large terminal time $T_\infty$ and using the limiting stationary solution there as an approximate boundary condition. In our implementation, we take $T_\infty = 3T$. The control process is considered on the interval $[0, T]$, while the $(\boldsymbol{H}(t), g_\gamma(t))$ pairs are numerically recovered by applying the Euler method [21] to integrate (4.5) backward over $[0, T_\infty]$. The limiting pair $(\boldsymbol{H}_\infty, g_{\gamma,\infty})$ is determined from the stationary version of (4.7), where $(\boldsymbol{H}, g_\gamma)$ is replaced by $(\boldsymbol{H}_\infty, g_{\gamma,\infty})$, and the time-dependent coefficients $\boldsymbol{R}$ and $\boldsymbol{Q}$ are replaced by their limiting values $\boldsymbol{R}_\infty$ and $\boldsymbol{Q}_\infty$.

**Periodic coefficients.** We now turn to the periodic setting, i.e., let the following coefficients be $P$-periodic for some $P > 0$:

$$\begin{aligned} &\boldsymbol{B}(t+P) = \boldsymbol{B}(t),\ \boldsymbol{\Sigma}(t+P) = \boldsymbol{\Sigma}(t),\ \boldsymbol{\alpha}(t+P) = \boldsymbol{\alpha}(t), \\ &\boldsymbol{\Lambda}(t+P) = \boldsymbol{\Lambda}(t),\ \boldsymbol{Q}(t+P) = \boldsymbol{Q}(t),\ \boldsymbol{R}(t+P) = \boldsymbol{R}(t). \end{aligned} \tag{4.8}$$

In this case, we seek the periodic solution $(\boldsymbol{H}, g_\gamma)$ of (4.4) -(4.5), with boundary conditions $\boldsymbol{H}(t+P) = \boldsymbol{H}(t)$, and $g_\gamma(t+P) = g_\gamma(t)$. Generally, iteration methods can be applied to solve the initial value $\boldsymbol{H}(0)$, and here we turn (4.5) into a shooting problem [8]. Then $\boldsymbol{H}(t)$ on $[0, P]$ can be calculated. Once the periodic function $\boldsymbol{H}(t)$ is determined, $g_\gamma$ can be constructed via $g_\gamma(t) = \frac{1}{1-e^{-\beta P}} \int_0^P e^{-\beta\tau} \Big( \mathrm{Tr}\big(\boldsymbol{\Sigma}(t+\tau)\boldsymbol{\Sigma}(t+\tau)^\top \boldsymbol{H}(t+\tau)\big) + \mathrm{Tr}(\boldsymbol{\Lambda}(t+\tau)\mathrm{diag}(\boldsymbol{\alpha}(t+\tau))\boldsymbol{H}(t+\tau)\mathrm{diag}(\boldsymbol{\alpha}(t+\tau))) + c_\gamma(t+\tau) \Big)\, \mathrm{d}\tau$.

We validate the method in two time-inhomogeneous settings: one with exponentially decaying coefficients and one with sinusoidally varying coefficients ($P = 10$); see Table 4 for detailed parameter choices. In both cases, we train for $N_{\text{itr}} = 3{,}000$ iterations on $[0, 20]$, using step size $\delta_t = 0.01$. The periodic case uses exploration intensity $\gamma = 0.05$ and the other case does not. Figure 3 compares the learned actor and critic with the benchmark solution: the learned mean control tracks the reference feedback

(panel (a)), the value trajectory aligns with the benchmark (panel (b)), and the state paths nearly coincide, including around jump times (panel (c)), demonstrating good agreement despite explicit time inhomogeneity and exploratory intensities.

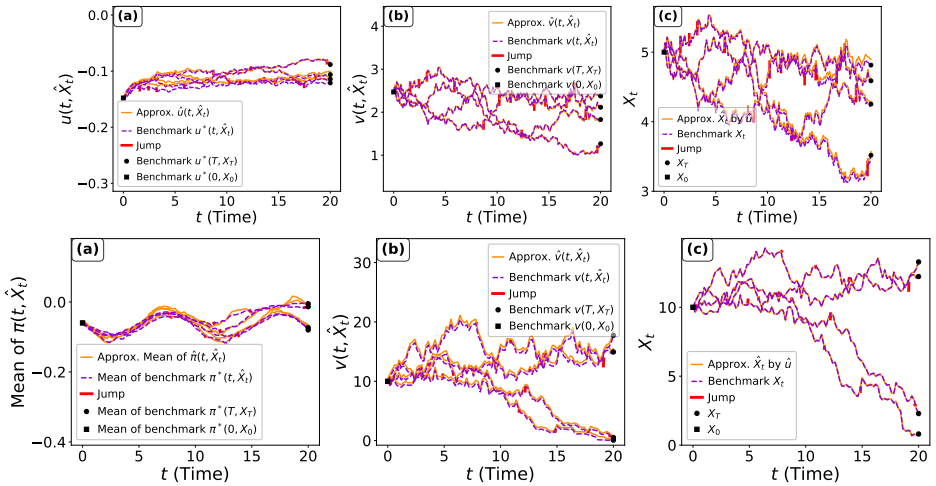



Fig. 3: Predicted (a) optimal control / mean of stochastic policy, (b) value function, and (c) state trajectory for time-inhomogeneous LQ problem on horizon $T = 20$: convergent coefficients (top) and periodic coefficients (bottom). Parameters: $N_{\text{itr}} =$ 3,000 and $\delta_t = 0.01$, with intensity $\gamma = 0$ (top) and $\gamma = 0.05$ (bottom).

**4.2. Merton Problem in a Jump-diffusion Market.** We consider Merton's portfolio optimization problem in a jump-diffusion market. The investor chooses a strategy $(u_t)_{t\geq 0}$ to allocate the fraction of the current wealth invested in the risky asset and of a risk-free asset with interest rate $r > 0$. The resulting controlled wealth process $(X_t)_{t\geq 0}$ satisfies

$$\mathrm{d}X_t = \big(r + u_t(\mu - r)\big)X_t\,\mathrm{d}t + \sigma u_t X_t\,\mathrm{d}W_t + \alpha u_t X_t\,\mathrm{d}M_t\,. \tag{4.9}$$

Suppose that the investor has a reward function $f(x) = \frac{x^p}{p}$ with $0 < p < 1$ (i.e., CRRA utility), and seeks to maximize the expected discounted reward. Since the model parameters $(r, \mu, \sigma, \lambda, \alpha)$ and the running reward $f$ are time-homogeneous, the problem admits a stationary solution.

**Standard Merton problem ($\gamma = 0$).** In this case, the optimal investment fraction $u^*$ can be solved by $(\mu - r) + (p-1)\sigma^2 u^* + \lambda\alpha\Big((1+\alpha u^*)^{p-1} - 1\Big) = 0$ provided that $1 + \alpha u^* > 0$, and the analytical value function $V(t,x) = V(x) = \frac{h^*}{p}x^p$, where $h^*$ satisfies $h^*\big[p\big(r+(\mu-r)u^*\big) + \frac{1}{2}p(p-1)\sigma^2(u^*)^2 + \lambda\big((1+\alpha u^*)^p - 1 - p\alpha u^*\big) - \beta\big] + 1 = 0$. Based on this analytical benchmark, Figure 4 compares the learned optimal control, value function, and state trajectories with the solution. Close agreement confirms that Alg. 3.1 accurately recovers the classical Merton solution in this jump–diffusion setting.

**Entropy-regularized Merton problem ($\gamma > 0$).** According to (2.4), the running reward is given by $\tilde{f}(x;\pi) = \frac{x^p}{p} + \gamma\mathcal{S}(\pi)$. The value function $V$ satisfies the entropy-regularized HJB equation in (2.10), which can be simplified as

$$V(x) = \frac{\gamma}{\beta}\log\int_{\mathcal{A}} \exp\Big(\tfrac{1}{\gamma}\,\mathscr{H}(x,u,V,\nabla_x V,\nabla_x^2 V)\Big)\,\mathrm{d}u \tag{4.10}$$

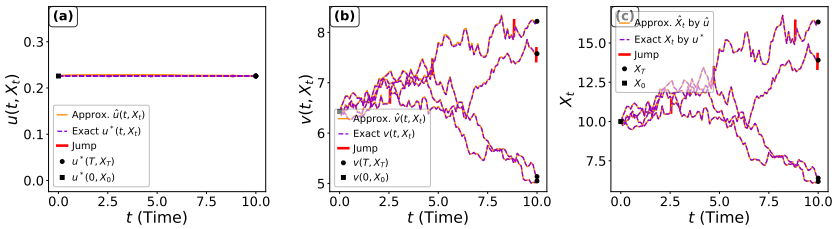



Fig. 4: Predicted (a) optimal control, (b) value function, and (c) state trajectories for the standard Merton problem ($\gamma = 0.0$) on horizon $T = 10$. Parameters: $\mu = 0.05$, $r = 0.03$, $\sigma = 0.4$, $\lambda = 0.2$, $\alpha = 0.3$, with total iterations $N_{\text{itr}} = 2{,}000$ and $\delta_t = 0.01$.

together with the Gibbs-type optimal policy $\pi^*(u \mid x) \propto \exp\big(\mathscr{H}/\gamma\big)$; see supplementary materials Section S.1.4 for a brief derivation. In general, the optimal policy $\pi^*$ is not Gaussian, since the Hamiltonian in the present Merton setting is not quadratic in $u$, unlike in the LQ case. As a result, under entropy regularization, the optimal policy is typically non-Gaussian, which leads to two main challenges: first, the policy distribution can no longer be accurately captured by a simple Gaussian family; second, in the absence of an explicit expression for the optimal policy, the value function $V$ generally does not admit a closed-form characterization. Fortunately, $V$ can be computed numerically to high accuracy via a physics-informed neural network (PINN) solver [34] applied to (4.10). The resulting numerical approximation of the value function can then be used to recover the corresponding optimal policy, thereby providing a benchmark solution. Full implementation details of the PINN solver are deferred to Appendix A.1.

With such a benchmark, our parameterization of $\pi_\theta$ using conditional normalizing flows, introduced in Section 3.3, provides a flexible framework for representing general non-Gaussian distributions. Figure 5 compares the conditional policy distributions at different time points, the value function, and the state trajectories. The parameter settings are reported in Table 4, while the trajectory construction is described in Appendix A.1. The close agreement with the PINN benchmark demonstrates the effectiveness of our algorithm and of the flow-based policy parameterization.

**4.3. Multi-Agent Portfolio Game in Jump-diffusion Market.** Our final example considers a game-theoretic extension of the Merton problem presented in Section 4.2, where each agent's reward depends on performance relative to the population average. This example serves two purposes: it demonstrates that the proposed algorithm scales well to high-dimensional systems and that it remains effective in the presence of strategic interactions.

We consider $n$ agents, indexed by $i \in \{1, \dots, n\}$, each choosing a control process $u_i$ to maximize their own expected discounted reward. The wealth process of agent $i$ evolves as

$$dX_t^i = u_i\big(b_i\,\mathrm{d}t + \eta_i\,\mathrm{d}W_t^i + \sigma_i\,\mathrm{d}W_t^0 + \alpha_i\,\mathrm{d}M_t^i + \xi_i\,\mathrm{d}M_t^0\big), \tag{4.11}$$

where $W^i$ is the idiosyncratic Brownian motion of agent $i$, $W^0$ is the common Brownian motion shared by all agents, $M^i$ is the compensated Poisson jump process specific to agent $i$, and $M^0$ represents common jump shocks. Fixing the strategies $\boldsymbol{u}_{-i}$ of all other agents, agent $i$ chooses $u_i$ to maximize

$$J^i(t, x, y; u_i, \boldsymbol{u}_{-i}) := \mathbb{E}\Big[\int_t^\infty e^{-\beta(s-t)}\, f_i\big(X_s^i, Y_s^i\big)\,\mathrm{d}s \;\Big|\; X_t^i = x,\ Y_t^i = y\Big], \tag{4.12}$$

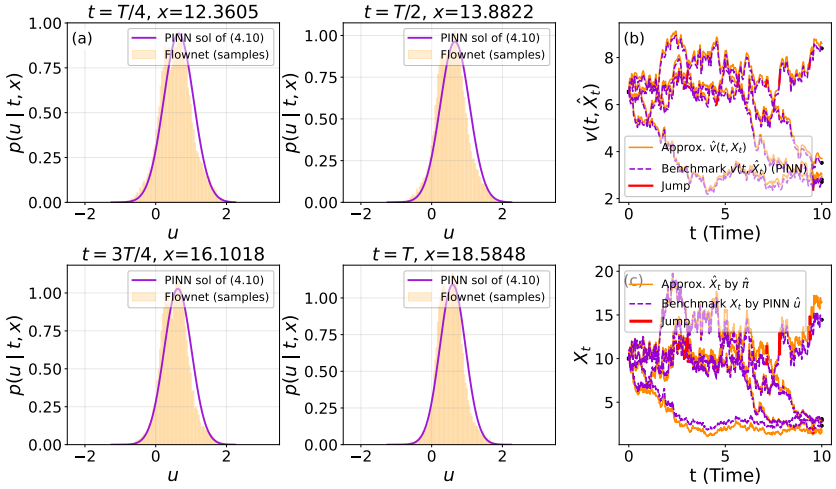



Fig. 5: Plots of (a) densities $p(u|\,t,\boldsymbol{x})$ of the stochastic policy $\pi(u|t,\boldsymbol{x})$ at $t = T/4$, $T/2$, $3T/4$, $T$ and some $\boldsymbol{x}$, (b) value function, and (c) state trajectories for the entropy-regularized Merton problem ($\gamma = 0.05$) on horizon $T = 10$. The total iterations is $N_{\text{itr}} = 2{,}000$ and $\delta_t = 0.05$.

where $Y_t^i := \frac{1}{n}\sum_{j\neq i} X_t^j$ represents the average wealth of the other agents, and reward $f_i(x,y) = -\exp\Big(-\frac{1}{\varrho_i}\Big(\big(1-\frac{\varpi_i}{n}\big)x - \varpi_i y\Big)\Big)$ measures relative performance, with risk-tolerance parameter $\varrho_i > 0$ and competition weight $\varpi_i \in \mathbb{R}$.

The goal of this multi-agent game is to find a Nash equilibrium, namely a collection of controls $\boldsymbol{u}^* = (u_1^*, \ldots, u_n^*)$ such that no agent can improve its own objective by deviating unilaterally while the others keep their strategies fixed. In other words, for every agent $i$, the control $u_i^*$ is an optimal response to $\boldsymbol{u}_{-i}^*$. To characterize such an equilibrium, we first solve the best-response problem of a representative agent and then impose consistency across all agents. Fixing the strategies $u_j$ of all agents $j \neq i$, we define the best-response value function of agent $i$ by

$$V^i(t,x,y) := \sup_{(u_i(s))_{s\geq t}} J^i(t,x,y;u_i,\boldsymbol{u}_{-i}). \tag{4.13}$$

For this portfolio game, the analytical benchmark is characterized by a coupled first-order system: the equilibrium investment strategy $\boldsymbol{u}^* = (u_1^*, \ldots, u_n^*)$ satisfies $\Psi_i'(u_i^*) = 0, i = 1, 2, \ldots, n$, where $\Psi_i(u) = -\chi_i b_i u + \frac{1}{2}\chi_i^2(\eta_i^2 + \sigma_i^2)u^2 - \chi_i\rho_i\sigma_i\widehat{u\sigma}\,u + \lambda_i\big(e^{-\chi_i\alpha_i u} - 1 + \chi_i\alpha_i u\big) + \lambda_0\Big(e^{-\chi_i\xi_i u + \rho_i\widehat{u\xi}} + \chi_i\xi_i u\Big)$, with $\widehat{u\sigma} := \frac{1}{n}\sum_{j\neq i} u_j\sigma_j$, $\widehat{u\xi} := \frac{1}{n}\sum_{j\neq i} u_j\xi_j$, $\chi_i := \frac{1-\varpi_i/n}{\varrho_i}$, $\rho_i := \frac{\varpi_i}{\varrho_i}$. Correspondingly, the value function of agent $i$ takes the form

$$V^i(x,y) = -\frac{1}{\beta - \Lambda_i^*}\exp\big(-\frac{1}{\varrho_i}\big(\big(1-\tfrac{\varpi_i}{n}\big)x - \varpi_i y\big)\big), \tag{4.14}$$

provided that $\beta > \Lambda_i^*$, where $\Lambda_i^* := \Psi_i(u_i^*) + C_i$, and $C_i = \rho_i\,\widehat{ub} + \frac{1}{2}\rho_i^2\big(\frac{1}{n^2}\sum_{j\neq i}(u_j\eta_j)^2 + \big(\frac{1}{n}\sum_{j\neq i}u_j\sigma_j\big)^2\big) + \sum_{j\neq i}\lambda_j\big(\exp\big(\frac{\varpi_i}{\varrho_i}\frac{u_j\alpha_j}{n}\big) - 1 - \frac{\varpi_i}{\varrho_i}\frac{u_j\alpha_j}{n}\big) - \lambda_0\,\rho_i\,\widehat{u\xi}$ is independent of $u_i$. Here $\widehat{ub} := \frac{1}{n}\sum_{j\neq i} u_j b_j$. The proof of the above characterization can be found in supplementary materials Section S.2.2; we use it as the analytical baseline when evaluating our numerical method.

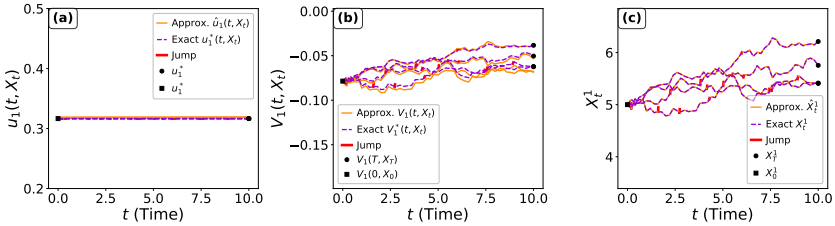



Fig. 6: Comparison of (a) optimal control, (b) value function and (c) state trajectories vs. benchmarks for agent 1 in the Merton portfolio game. Results are shown after $N_{\text{itr}} = 1,000$ training iterations with $\delta_t = 0.02$.

Table 3: Runtime and last-iteration relative errors for different (number of time points, number of agents) pairs $= (K, n)$ of our algorithm for multi-agent game.

| | | $n$=2 | $n$=5 | $n$=10 | $n$=25 |
|---|---|---|---|---|---|
| $K = 100$ | Runtime (min) | 18.98 | 40.67 | 76.03 | 205.76 |
| | $\mathcal{E}_V$ | 0.1853 | 0.1866 | 0.1490 | 0.1767 |
| | $\mathcal{E}_{\text{u}}$ | 0.0300 | 0.0201 | 0.0330 | 0.0318 |
| $K = 500$ | Runtime (min) | 94.72 | 202.19 | 400.77 | 1050.64 |
| | $\mathcal{E}_V$ | 0.0617 | 0.0355 | 0.0447 | 0.0476 |
| | $\mathcal{E}_{\text{u}}$ | 0.0081 | 0.00137 | 0.0176 | 0.0261 |

Figures 6–7 illustrate the numerical results with $n = 25$ agents. During training, each agent has its own actor and critic networks, and all agents can be trained in parallel. At every iteration each agent updates its parameters against the other agents' policies from the previous iteration, aiming to converge to the Nash equilibrium. We consider a heterogeneous setting where agent 1 faces a different market and preference parameters, while the rest are homogeneous. The full parameter settings are reported in Appendix A Table 4. All plots are generated after $N_{\text{itr}} = 1,000$ training iterations with time step $\delta_t = 0.02$. Figure 6 depicts the predicted control, value function, and state trajectories for agent 1. Figure 7 displays the RMSEs of the control and the value function for each agent, together with the averaged training loss of all agents. Overall, the close alignment with the benchmarks, together with the small RMSEs and stable training loss, indicates that our method achieves promising and stable performance even in high dimensions and in the presence of strategic interactions.

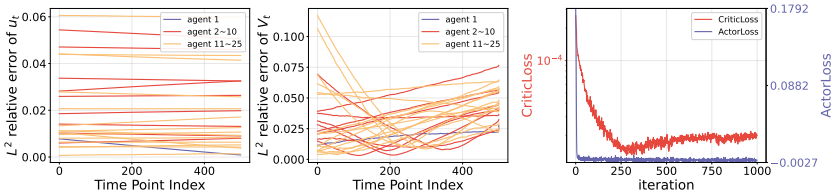



Fig. 7: MSEs of the learned controls and value functions for each agent ($n = 25$), together with the training loss averaged over all agents

. The full parameter setting is reported in Table 4.

Table 3 reports the runtime and RMSEs (Eqs. (4.1)–(4.2)) for different choices of time step $\delta_t$ and number of agents $n$. The runtime increases approximately linearly with $n$, indicating favorable scalability with respect to the problem dimension. The errors remain comparable across these configurations, indicating stable performance as the problem size grows.

**5. Conclusions and Discussions.** This paper develops a reinforcement learning framework for infinite-horizon time-inhomogeneous stochastic control problem subject to jump-diffusion and entropy regularization. We introduce a continuous-time little $q$-function, define an appropriate time-dependent occupation measure, and establish its structural properties. This representation leads to a general policy gradient formula, and we design an actor-critic algorithm tailored to general time-inhomogeneous jump-diffusion dynamics and non-Gaussian stochastic policies via conditional normalizing flow. We also derive explicit solutions for the value function and the optimal (stochastic) policy for several canonical specifications, including LQ control, the Merton portfolio problem, and multi-agent portfolio game with CARA utilities in jump-diffusion markets. These closed-form characterizations provide ground-truth benchmarks for evaluating RMSEs of the proposed algorithm. Our method is validated on a suite of low- and high-dimensional experiments, including settings with jump components and time-dependent coefficients. Across multiple evaluation metrics, the learned policies closely track the analytic solutions when available, and our algorithm exhibits strong performance broadly.

There are several natural directions for future work. Building on the proposed actor-critic framework, it would be interesting to study how alternative entropy regularizers affect the control problem, for example the Tsallis entropy considered in [9]. However, its structure typically precludes closed-form optimal stochastic policies, making benchmarking more challenging. On the theoretical side, extending the little $q$-framework and occupation measure to partially observed models or mean-field interaction structures would further bridge continuous-time RL and modern stochastic control with jumps. On the algorithmic side, scaling our approach to very high-dimensional problems and integrating it with large language models or agent-based architectures are promising directions; recent progress on agent-based methods and DFA-type accelerators suggests substantial potential for speeding up RL in complex control environments [1, 2].

**References.**

[1] Shayan Meshkat Alsadat, Jean-Raphaël Gaglione, Daniel Neider, Ufuk Topcu, and Zhe Xu. Using large language models to automate and expedite reinforcement learning with reward machine. In *2025 American Control Conference (ACC)*, pages 206–211. IEEE, 2025.

[2] Shayan Meshkat Alsadat and Zhe Xu. Multi-agent reinforcement learning in non-cooperative stochastic games using large language models. *IEEE Control Systems Letters*, 8:2757–2762, 2024.

[3] David Applebaum. *Lévy Processes and Stochastic Calculus*. Cambridge University Press, 2009.

[4] Burcu Aydoğan and Mogens Steffensen. Optimal investment strategies under the relative performance in jump-diffusion markets. *Decisions in Economics and Finance*, 48(1):179–204, 2025.

[5] Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, 5:834–846, 2012.

[6] Christian Bender and Nguyen Tran Thuan. Entropy-regularized mean-variance portfolio optimization with jumps. *arXiv preprint arXiv:2312.13409*, 2023.

[7] Christian Bender and Nguyen Tran Thuan. Continuous time reinforcement learning: A random measure approach. *Stochastic Processes and their Applications*, 194:104848, 2026.

[8] Sergio Bittanti, Patrizio Colaneri, and Giuseppe De Nicolao. The periodic riccati equation. In *The Riccati Equation*, pages 127–162. Springer, 1991.

[9] Lijun Bo, Yijie Huang, Xiang Yu, and Tingting Zhang. Continuous-time q-learning for jump-diffusion models under tsallis entropy. *arXiv preprint arXiv:2407.03888*, 2024.

[10] Janaka Brahmanage, Jiajing Ling, and Akshat Kumar. Flowpg: action-constrained policy gradient with normalizing flows. *Advances in Neural Information Processing Systems*, 36:20118–20132, 2023.

[11] Wei Cai, Shuixin Fang, Wenzhong Zhang, and Tao Zhou. Martingale deep learning for very high dimensional quasi-linear partial differential equations and stochastic optimal controls. *arXiv preprint arXiv:2408.14395*, 2024.

[12] Patrick Cheridito, Jean-Loup Dupret, and Donatien Hainaut. Deep learning for continuous-time stochastic control with jumps. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[13] Min Dai, Yuchao Dong, and Yanwei Jia. Learning equilibrium mean-variance strategy. *Mathematical Finance*, 33(4):1166–1212, 2023.

[14] Robert Denkert, Huyên Pham, and Xavier Warin. Control randomisation approach for policy gradient and application to reinforcement learning in optimal switching. *Applied Mathematics & Optimization*, 91(1):9, 2025.

[15] Kenji Doya. Reinforcement learning in continuous time and space. *Neural Computation*, 12(1):219–245, 2000.

[16] Jinqiao Duan. *An Introduction to Stochastic Dynamics*. Cambridge University Press, Cambridge, 2015.

[17] Conor Durkan, Artur Bekasov, Iain Murray, and George Papamakarios. Neural spline flows. *Advances in Neural Information Processing Systems*, 32, 2019.

[18] Xuefeng Gao, Lingfei Li, and Xun Yu Zhou. Reinforcement learning for jump-diffusions, with financial applications. *Mathematical Finance*, 2026.

[19] Xin Guo, Anran Hu, and Yufei Zhang. Reinforcement learning for linear-convex models with jumps via stability analysis of feedback controls. *SIAM Journal on Control and Optimization*, 61(2):755–787, 2023.

[20] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

[21] Ernst Hairer, Gerhard Wanner, and Syvert P Nørsett. *Solving ordinary differential equations I: Nonstiff problems*. Springer, 1993.

[22] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[23] Yanwei Jia, Du Ouyang, and Yufei Zhang. Accuracy of discretely sampled stochastic policies in continuous-time reinforcement learning. *arXiv preprint arXiv:2503.09981*, 2025.

[24] Yanwei Jia and Xun Yu Zhou. Policy evaluation and temporal-difference learning in continuous time and space: A martingale approach. *Journal of Machine Learning Research*, 23(154):1–55, 2022.

[25] Yanwei Jia and Xun Yu Zhou. Policy gradient and actor-critic learning in continuous time and space: Theory and algorithms. *Journal of Machine Learning Research*, 23(275):1–50, 2022.

[26] Yanwei Jia and Xun Yu Zhou. q-learning in continuous time. *Journal of Machine Learning Research*, 24(161):1–61, 2023.

[27] Chenyang Jiang, Donggyu Kim, Alejandra Quintos, and Yazhen Wang. Robust reinforcement learning under diffusion models for data with jumps. *arXiv preprint arXiv:2411.11697*, 2024.

[28] Liwei Lu, Ruimeng Hu, Xu Yang, and Yi Zhu. Multiagent relative investment games in a jump diffusion market with deep reinforcement learning algorithm. *SIAM Journal on Financial Mathematics*, 16(2):707–746, 2025.

[29] Robert C. Merton. Optimum consumption and portfolio rules in a continuous-time model. *Journal of Economic Theory*, 3(4):373–413, 1971.

[30] Alessandro Montenegro, Marco Mussi, Alberto Maria Metelli, and Matteo Papini. Learning optimal deterministic policies with stochastic policy gradients. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 36160–36211. PMLR, 2024.

[31] Bernt Øksendal and Agnes Sulem. *Applied Stochastic Control of Jump Diffusions*, volume 3. Springer, 2007.

[32] George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. Normalizing flows for probabilistic modeling and inference. *Journal of Machine Learning Research*, 22(57):1–64, 2021.

[33] Huyên Pham. *Continuous-Time Stochastic Control and Optimization with Financial Applications*, volume 61. Springer Science & Business Media, 2009.

[34] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[35] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. Poster.

[36] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in Neural Information Processing Systems*, 12, 1999.

[37] Lukasz Szpruch, Tanut Treetanthiploet, and Yufei Zhang. Optimal scheduling of entropy regularizer for continuous-time linear-quadratic reinforcement learning. *SIAM Journal on Control and Optimization*, 62(1):135–166, 2024.

[38] Corentin Tallec, Léonard Blier, and Yann Ollivier. Making deep q-learning methods robust to time discretization. In *International Conference on Machine Learning*, pages 6096–6104. PMLR, 2019.

[39] Haoran Wang, Thaleia Zariphopoulou, and Xun Yu Zhou. Reinforcement learning in continuous time and space: A stochastic control approach. *Journal of Machine Learning Research*, 21(198):1–34, 2020.

[40] Haoran Wang and Xun Yu Zhou. Continuous-time mean–variance portfolio selection: A reinforcement learning framework. *Mathematical Finance*, 30(4):1273–1308, 2020.

[41] Xiaoli Wei and Xiang Yu. Continuous time q-learning for mean-field control problems. *Applied Mathematics & Optimization*, 91(1):10, 2025.

[42] Hanyang Zhao, Wenpin Tang, and David Yao. Policy optimization for continuous reinforcement learning. *Advances in Neural Information Processing Systems*, 36:13637–13663, 2023.

[43] Mo Zhou, Jiequn Han, and Jianfeng Lu. Actor-critic method for high dimensional

static Hamilton–Jacobi–Bellman partial differential equations based on neural networks. *SIAM Journal on Scientific Computing*, 43(6):A4043–A4066, 2021.

[44] Mo Zhou, Haosheng Zhou, and Ruimeng Hu. Learning mean-field games through mean-field actor-critic flow. *arXiv preprint arXiv:2510.12180*, 2025.

## Appendix A. More Numerical Details.

### A.1. Neural Network (NN) Architectures and Experimental Details.

**Critic network.** We parameterize the critic $V_\psi : (t, \boldsymbol{x}) \mapsto \mathbb{R}$ by a ResNet [22] of depth 3, with input dimension $d+1$, hidden width $d+10$, `tanh` activations, and a scalar linear readout. All critic networks are optimized with Adam (learning rate $10^{-3}$) and a fixed random seed (2025). For the LQ cases and the multi-player game we decay the learning rate with a multi-step schedule; for the Merton problem with flow layers we use a `CosineAnnealingWarmUp` schedule.

**Actor network.** We parameterize the actor $\pi_\theta(\cdot \mid t, \boldsymbol{x})$ as a conditional normalizing flow. Given $(t, \boldsymbol{x})$, a ResNet with `tanh` activations, depth 3, and hidden width $d + 10$ outputs the parameters of the Gaussian base distribution in (3.19). For the standard case with $\gamma = 0$, we use a fixed standard deviation $\mathrm{Std} = 0.1$ instead of a learnable $\mathrm{Std}_\theta$. This follows [30, 39], where fixing the exploration scale is found to improve stability when stochastic policies are used to learn an underlying deterministic optimal control, while still being sufficient for convergence. This applies to all LQ cases and the multi-player Merton game. The actor is optimized with Adam using learning rate $10^{-3}$ for time-homogeneous problems and $5 \times 10^{-4}$ for time-inhomogeneous problems, together with a multi-step learning-rate schedule.

For the entropy-regularized Merton problem, samples from the Gaussian base distribution are further transformed by an invertible flow map $F_\theta$. Specifically, we employ a conditional monotone rational-quadratic spline coupling [17] on $\mathbb{R}^m$ for $F_\theta$, where each coordinate transformation is represented by a piecewise rational-quadratic spline. We use $K_{\mathrm{bin}} = 6$ bins with identity tails outside $[-B_{\mathrm{flow}}, B_{\mathrm{flow}}] = [-2.5, 2.5]$. The spline parameters are produced by a ResNet with `tanh` activations, depth 2, and hidden width 32. To improve stability in early training, we initialize the flow close to the identity map and freeze its parameters for the first 30 actor updates. After this warm-up stage, the flow is applied directly, i.e., $\boldsymbol{z}_K = F_\theta(\boldsymbol{z}_0)$. Bounded actions are enforced through a temperature-controlled sigmoid squashing function $\boldsymbol{u} = S(\boldsymbol{z}_K; \tau)$. We anneal $\tau$ from 2.0 to 1.0 over the first 30 squash-delay steps so that the squashing is milder during warm-up and actions are less likely to saturate near the boundaries.

When the flow $F_\theta$ is enabled, the actor's base network and the spline flow layers are trained using an Adam optimizer with two parameter groups. The base network uses $6 \times 10^{-5}$, while the flow layers use learning rate $1 \times 10^{-5}$. We choose a smaller learning rate for the flow because the spline–squash composition is more sensitive to parameter updates, and a lower step size improves training stability. Both parameter groups are scheduled jointly by a single `CosineAnnealingWarmUp` scheduler.

**PINN approach for Merton's problem.** We compute a reference solution by solving simplified HJB equation (4.10) on uniform grids $x_{\mathrm{grid}} \subset [x_{\min}, x_{\max}]$ and $u_{\mathrm{grid}} \subset [u_{\min}, u_{\max}]$, with $n_x = 500$ grid points for $x$ and $n_u = 400$ grid points for $u$. The value function in PINN approach is parameterized by a feedforward NN $V_\phi(x)$ with 5 hidden layers, each of width 256, and `tanh` activations. All first- and second-order derivatives of $V_\phi$ entering the Hamiltonian $\mathscr{H}(x, u, V_\phi, \nabla_x V_\phi, \nabla_x^2 V_\phi)$ are computed via automatic differentiation. To approximate the integral in (4.10), we use the trapezoidal quadrature on $u_{\mathrm{grid}}$, denoted by $\mathcal{Q}_u[\cdot]$. For each $(x_i, u_j)$, let $H_{ij} = \mathscr{H}\big(x_i, u_j, V_\phi(x_i), \nabla_x V_\phi(x_i), \nabla_x^2 V_\phi(x_i)\big)$. Then the right-hand side of (4.10)

is approximated by $V_{\mathrm{rhs}}(x_i) = \frac{\gamma}{\beta}\log\big(\mathcal{Q}_u[\exp(H_{ij}/\gamma)]\big)$. In practice, for numerical stability, we evaluate it through a log-sum-exp implementation of the quadrature. The network parameters $\phi$ are trained by minimizing the mean squared residual $\mathcal{L}_{\mathrm{PINN}}(\phi) = \frac{1}{n_x}\sum_{i=1}^{n_x}\big(V_\phi(x_i) - V_{\mathrm{rhs}}(x_i)\big)^2$, using Adam with learning rate $5\times 10^{-4}$ for $2,000$ iterations.

After obtaining the PINN solution of the value function, we recover the optimal policy on the grid $u_{\mathrm{grid}}$ as the Gibbs distribution induced by the same quadrature normalization, $\pi_{ij} = \frac{w_j \exp(H_{ij}/\gamma)}{\sum_{k=1}^{n_u} w_k \exp(H_{ik}/\gamma)}$, where $\{w_j\}_{j=1}^{n_u}$ are the trapezoidal weights.

**A.2. Parameter Setting.** Table 4 records parameters in our experiments.

Table 4: Parameter settings in numerical experiments.

| | **Time-inhomogeneous LQ** | | **LQ (Homo.)** | **Merton** | | **Multi-Agent CARA** |
|---|---|---|---|---|---|---|
| | **Convergent** | **Periodic** | | **Standard** | **Entropy** | |
| Dim / agent | $d = 1$ | | $d = 1, 5, 20, 50$ | $d = 1$ | | $n = 25$ |
| Drift coeff. | $B_t u_t = b(t)\, u_t$,<br>$b(t) = b_\infty + (b_0 - b_\infty)e^{-\kappa_b t}$<br>$b_0 = 0.6,\ b_\infty = 0.5,\ \kappa_b = 1.0$ | $B_t u_t = b(t)\, u_t$,<br>$b(t) = b_{\text{bar}} + b_{\text{amp}} \sin\left(\frac{2\pi t}{P_b} + \phi_b\right)$<br>$b_{\text{bar}} = 0.12,\ b_{\text{amp}} = 0.06,$<br>$P_b = 10.0,\ \phi_b = 0.0$ | $B\mathbf{u}_t = 0.5\,\mathbf{I}\,\mathbf{u}_t$ | $(r + u_t(\mu - r))X_t$<br>$\mu = 0.05 \quad \mu = 0.1$<br>$r = 0.03 \quad r = 0.05$ | | $b_i = \begin{cases} 0.05, & i = 1, \\ 0.02, & 2 \le i \le n \end{cases}$ |
| Diffusion coeff. | $\Sigma_t = \sigma(t)$,<br>$\sigma(t) = \sigma_\infty + (\sigma_0 - \sigma_\infty)e^{-\kappa_\sigma t}$<br>$\sigma_0 = 0.3,\ \sigma_\infty = 0.2,\ \kappa_\sigma = 1.0$ | $\Sigma_t = \sigma(t)$,<br>$\sigma(t) = \sigma_{\text{bar}} + \sigma_{\text{amp}} \sin\left(\frac{2\pi t}{P_\sigma} + \phi_\sigma\right)$<br>$\sigma_{\text{bar}} = 0.2,\ \sigma_{\text{amp}} = 0.1,$<br>$P_\sigma = 10.0,\ \phi_\sigma = 0.0$ | $\mathbf{\Sigma} = 0.3\,\mathbf{I}$ | $\sigma = 0.4$ | | $\eta_i = \begin{cases} 0.08, & i = 1, \\ 0.05, & 2 \le i \le n \end{cases}$<br>$\sigma_i = \begin{cases} 0.5, & i = 1, \\ 0.4, & 2 \le i \le n \end{cases}$ |
| Jump intensity | $\lambda_t \equiv \lambda,\ \lambda = 0.2$ | | $\boldsymbol{\lambda} = \text{linspace}(0.2, 0.3, D)$ | $\lambda = 0.2$ | $\lambda = 0.3$ | $\lambda_i \equiv 0.2$<br>$\lambda_0 = 0.25$ |
| Jump sizes / coeff. | $\alpha_t = \alpha(t)$,<br>$\alpha(t) = \alpha_\infty + (\alpha_0 - \alpha_\infty)e^{-\kappa_\alpha t}$<br>$\alpha_0 = 0.3,\ \alpha_\infty = 0.2,\ \kappa_\alpha = 1.0$ | $\alpha_t = \alpha(t)$,<br>$\alpha(t) = \alpha_{\text{bar}} + \alpha_{\text{amp}} \sin\left(\frac{2\pi t}{P_\alpha} + \phi_\alpha\right)$<br>$\alpha_{\text{bar}} = 0.2,\ \alpha_{\text{amp}} = 0.1,$<br>$P_\alpha = 10.0,\ \phi_\alpha = 0.0$ | $\boldsymbol{\alpha} = \text{linspace}(0.3, 0.2, D)$ | $\alpha = 0.3$ | $\alpha = 0.1$ | $\alpha_i \equiv 0.2$<br>$\xi_i \equiv 0.2$ |
| Discount factor | $\beta = 1.0$ | | $\beta = 1.0$ | $\beta = 1.0$ | | $\beta = 1.0$ |
| Exploration intensity $\gamma$ | $\gamma = 0.0$ | $\gamma = 0.05$ | $\gamma = 0$ or $0.05$ | $\gamma = 0$ | $\gamma = 0.05$ | $\gamma = 0$ |
| Learning rate (actor) | $5 \times 10^{-4}$ | | $10^{-3}$ | $10^{-3}$ | $6 \times 10^{-5}$ | $10^{-3}$ |
| Learning rate (critic) | $10^{-3}$ | | $10^{-3}$ | $10^{-3}$ | | $10^{-3}$ |
| Iterations | $N_{\text{itr}} = 3000$ | | 1,000 | 2,000 | 2,000 | 1,000 |
| Time step size / $\delta_t$ | $T = 20,\ K = 2000,\ \delta_t = T/K = 0.01$ | | $\delta_t = 0.01$ | $\delta_t = 0.01$ | $\delta_t = 0.05$ | $\delta_t = 0.02$ |
| Minibatch size | $L = 100$ | | $L = 100$ | $L = 500$ | $L = 200$ | $L = 100$ |
| Note | $\boldsymbol{R} = 2\boldsymbol{I}_d,\ \boldsymbol{Q} = 0.1\boldsymbol{I}_d$<br>$K_{\text{actor}} = 15,\ K_{\text{critic}} = 5$ | | $\boldsymbol{R} = 5\boldsymbol{I}_d,\ \boldsymbol{Q} = 0.5\boldsymbol{I}_d$<br>$K_{\text{actor}} = 20,\ K_{\text{critic}} = 5$ | $p = 0.5$<br>$K_{\text{actor}} = 20,\ K_{\text{critic}} = 5$ | | $\varpi_i \equiv 0.2$<br>$\varrho_i = \begin{cases} 1.5, & i = 1, \\ 2.0, & 2 \le i \le n \end{cases}$<br>$K_{\text{actor}} = 30,\ K_{\text{critic}} = 10$ |
| Riccati reference solve | $N_{\text{riccati}} = 3000,\ \text{d}t_{\text{ODE}} = 10^{-3},\ T_{\text{trunc}} = 3T$ | | – | – | | – |